

CSE 4303

Data Structures

Topic: Topological Sort

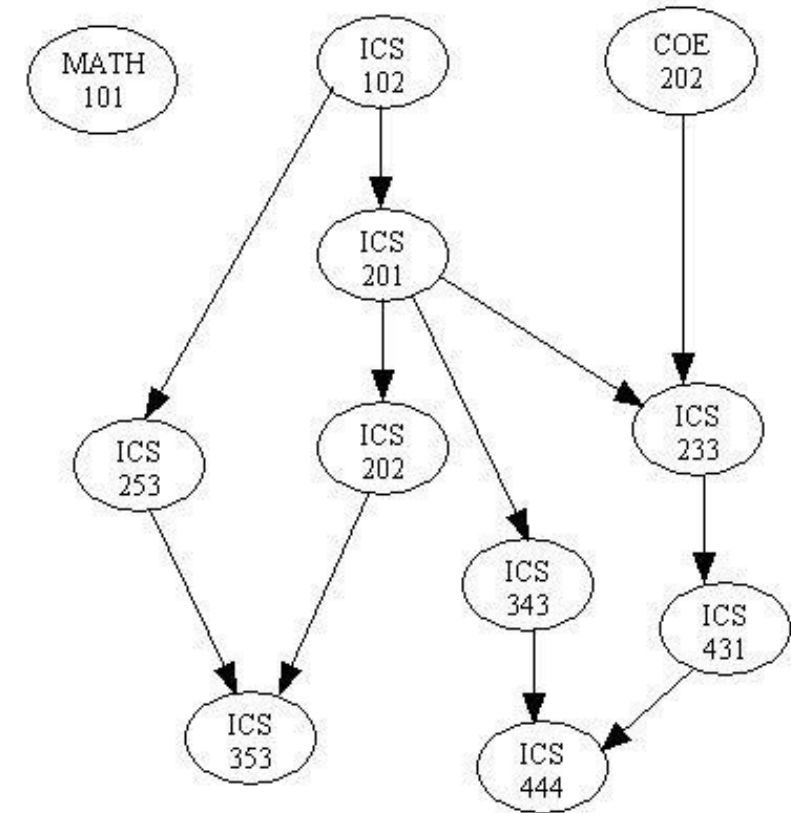
Sabbir Ahmed

Assistant Prof | CSE | IUT

sabbirahmed@iut-dhaka.edu

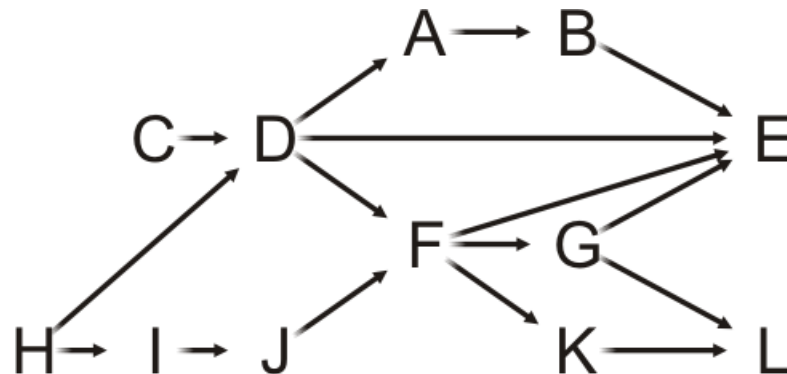
Motivation

- Given a set of **tasks with dependencies**:
 - Find is there an **order** in which we can complete the tasks?
- Cycles in dependencies can cause issues...



Definition

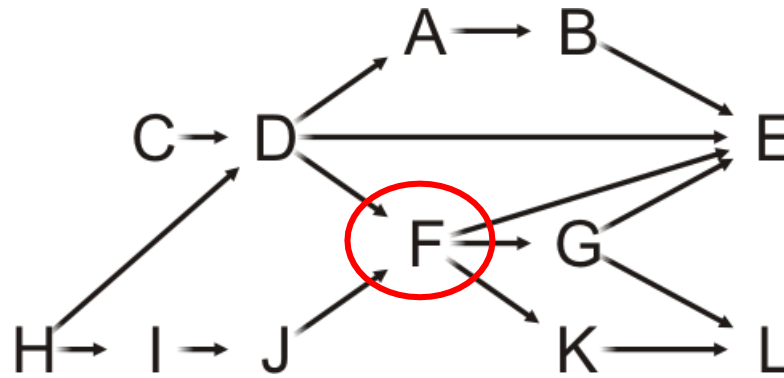
- Topological sorting of the vertices in a DAG:
 - An ordering $v_1, v_2, v_3, \dots, v_{|V|}$ such that v_j appears before v_k if there is a path from v_j to v_k .
- Given this DAG, a topological sort is
 $H, C, I, D, J, A, F, B, G, K, E, L$



Example

- There are paths from H, C, I, D, J to F , so all these must come before F in a topological sort

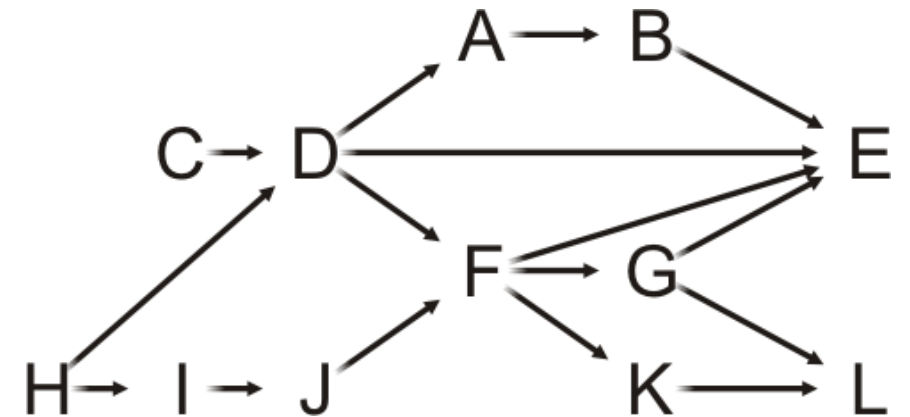
$H, C, I, D, J, A, F, B, G, K, E, L$



Clearly, this sorting need not be unique

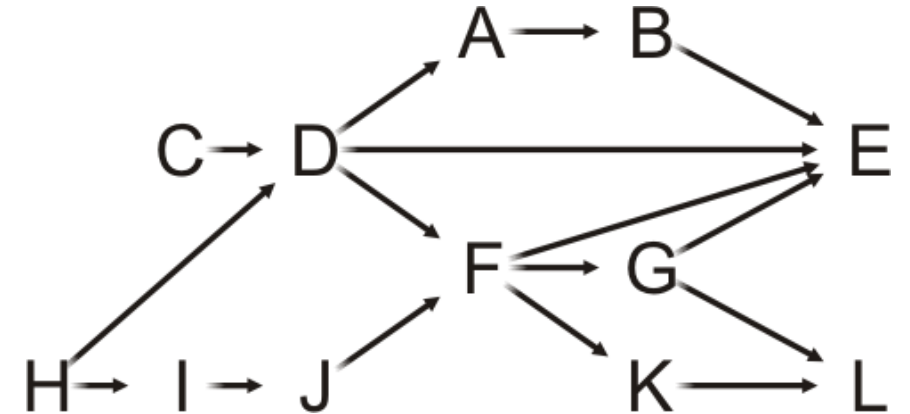
Topological Sort

- **Idea:**
- Given a DAG V , make a copy W and iterate:
 - Find a vertex v in W with **in-degree zero**
 - Let v be the next vertex in the topological sort
 - Continue iterating with the vertex-induced subgraph $W \setminus \{v\}$



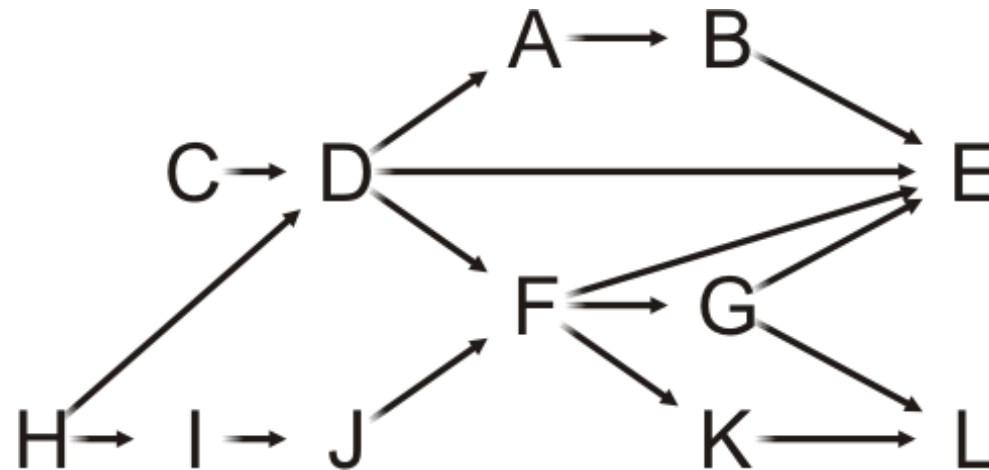
Topological Sort

- **Idea:**
- Given a DAG V , make a copy W and iterate:
 - Find a vertex v in W with **in-degree zero**
 - Let v be the next vertex in the topological sort
 - Continue iterating with the vertex-induced sub-graph $W \setminus \{v\}$
- On this graph, iterate the following $|V| = 12$ times
 - Choose a vertex v **that has in-degree zero**.
 - Let v be the next vertex in our topological sort
 - Remove v and all edges connected to it

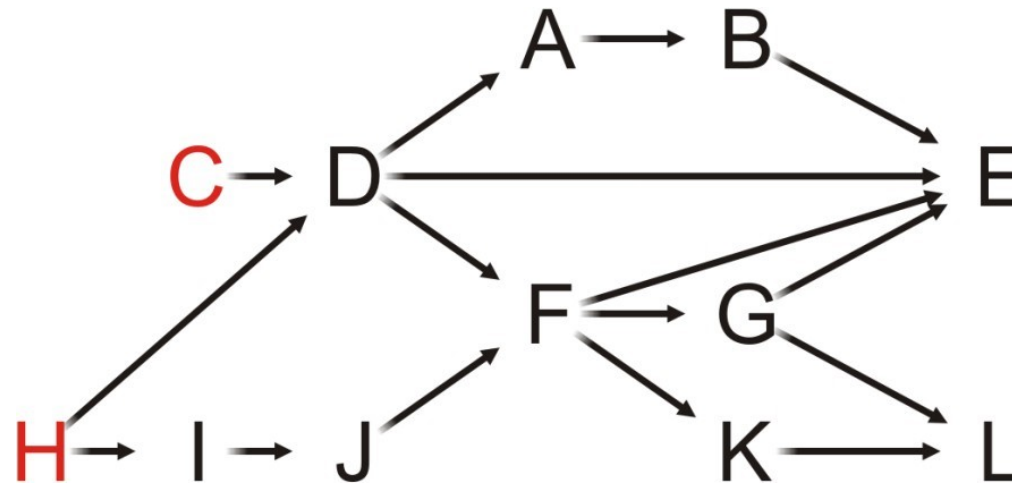


Example

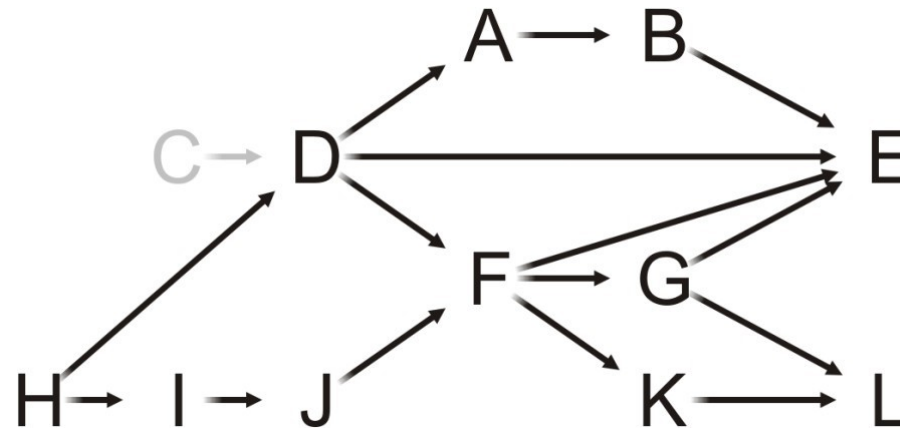
- Let's step through this algorithm with this example
 - Which task can we start with?



- Out of Tasks C or H, let's choose Task C
(we could also start with H, hence Topological order need not be unique)

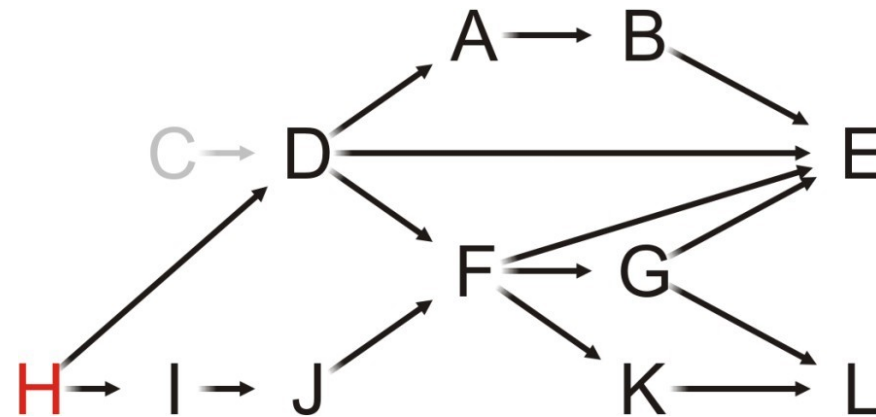


- Having completed Task C, which vertices have in-degree zero?



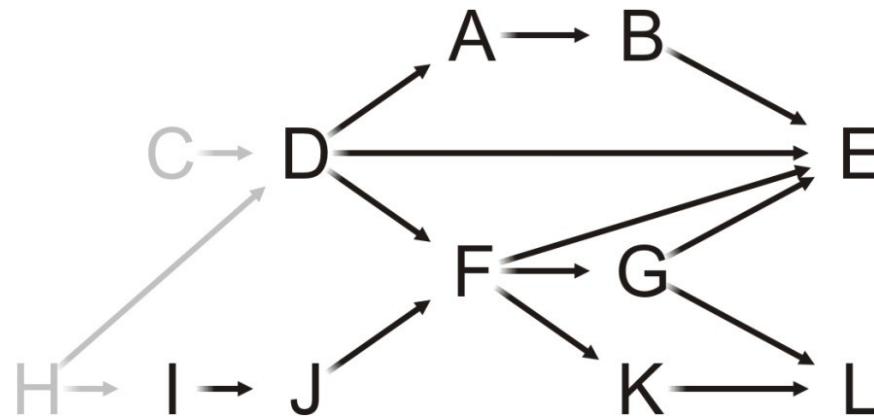
C

- Only Task H can be completed, so we choose it



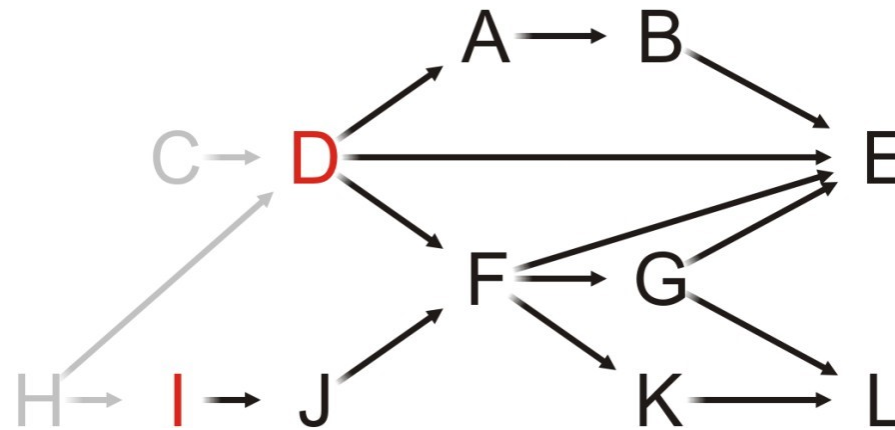
C

- Having removed H, what is next?



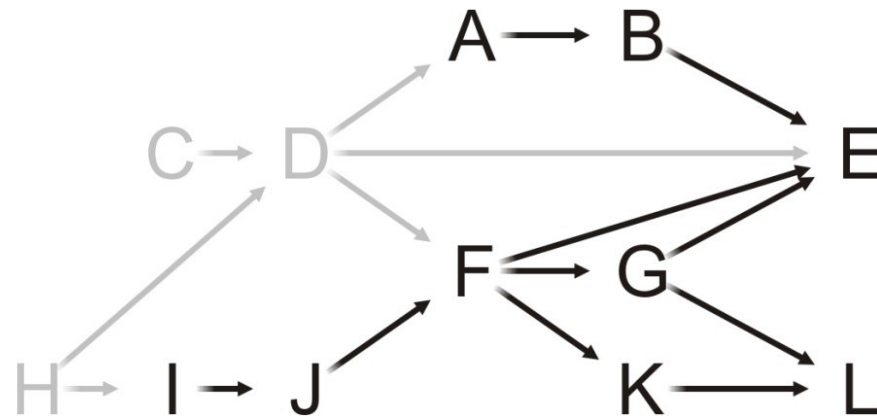
C, H

- Both Tasks D and I have in-degree zero
- Let us choose Task D



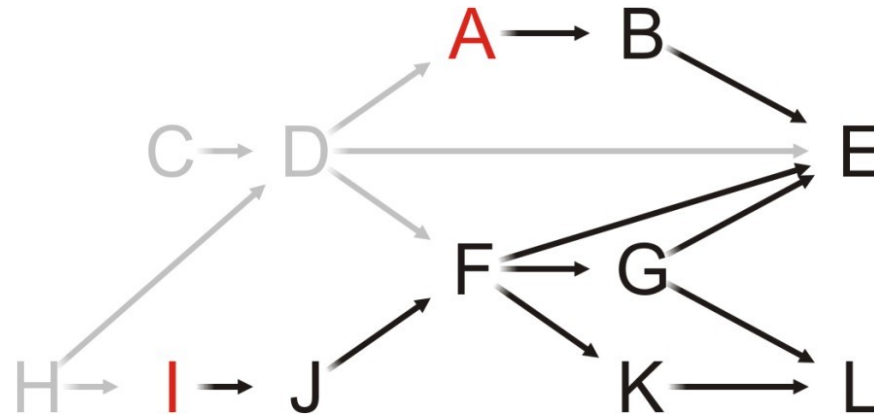
C, H

- We remove Task D, and now?



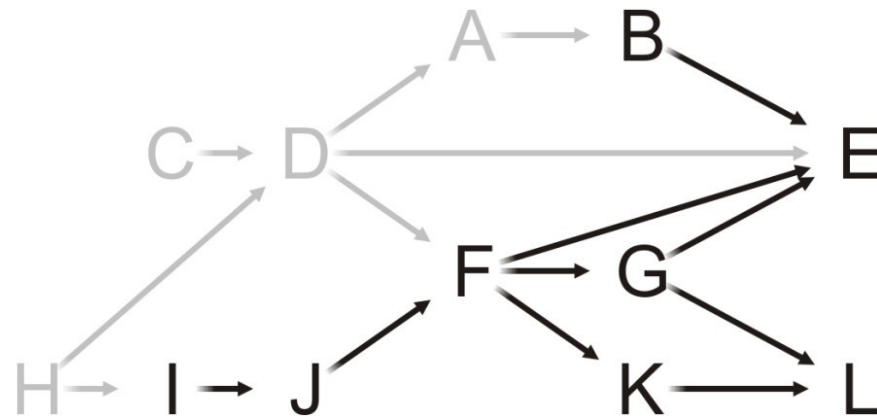
C, H, D

- Both Tasks A and I have in-degree zero
- Let's choose Task A



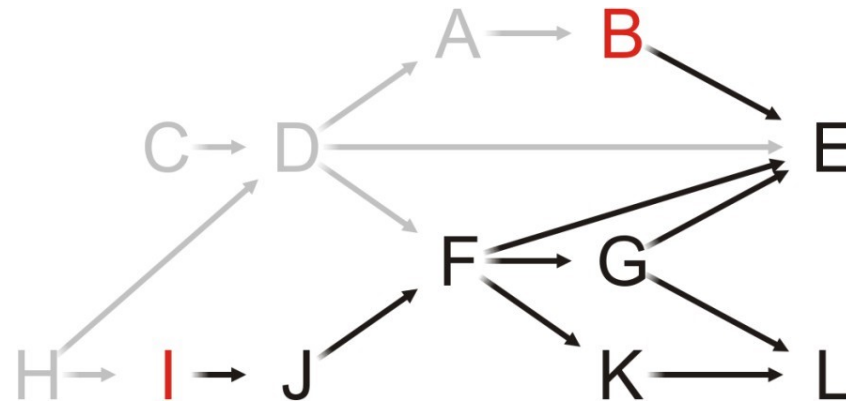
C, H, D

- Having removed A, what now?



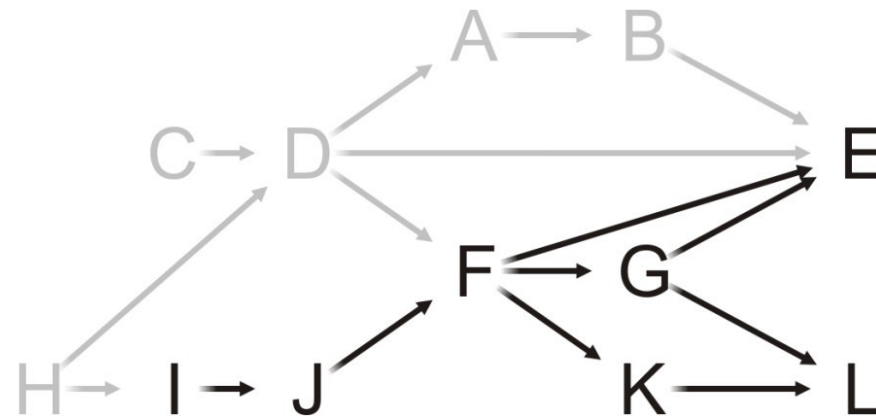
C, H, D, A

- Both Tasks B and I have in-degree zero
- Choose Task B



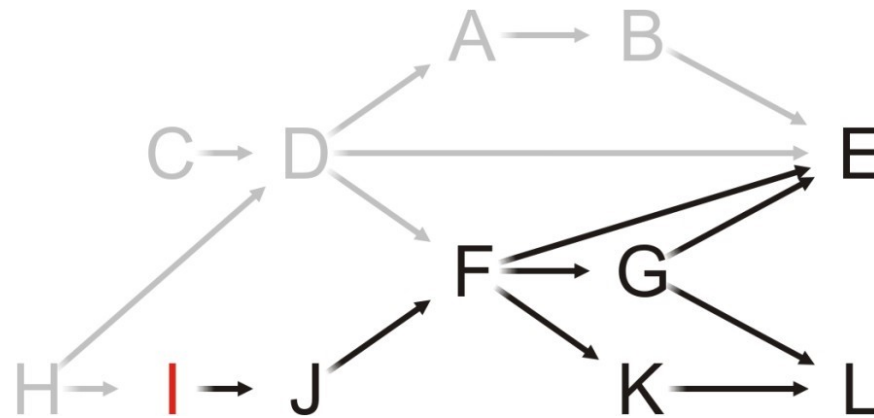
C, H, D, A

- Removing Task B, we note that Task E still has an in-degree of two
- Next?



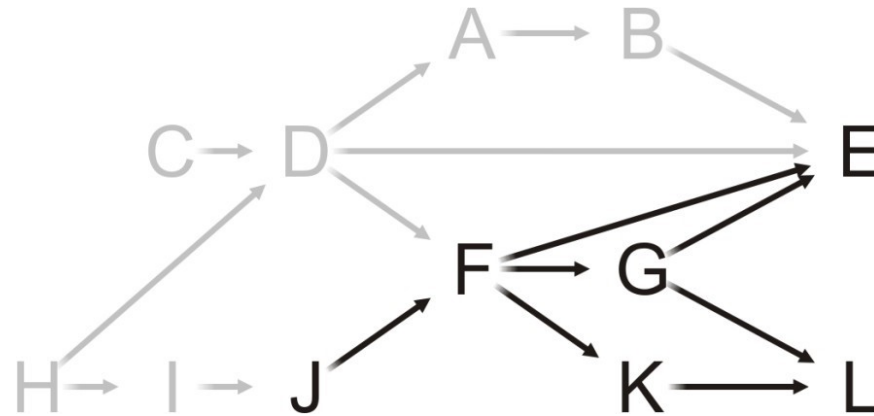
C, H, D, A, B

- As only Task I has in-degree zero, we choose it



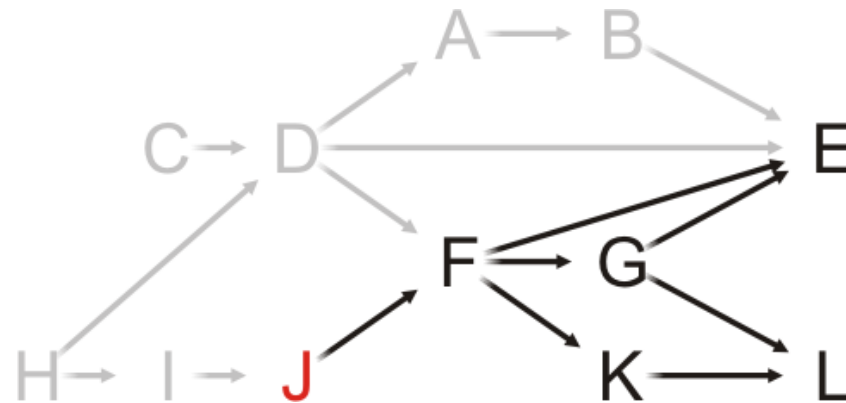
C, H, D, A, B

- Having completed Task I, what now?



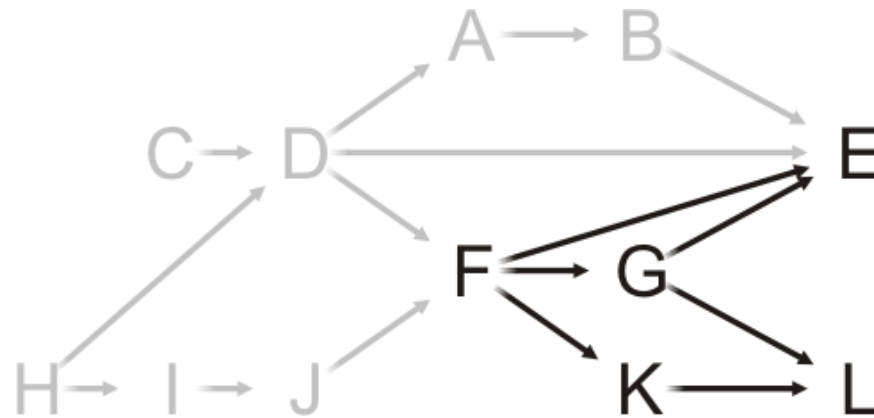
C, H, D, A, B, I

- Only Task J has in-degree zero: choose it



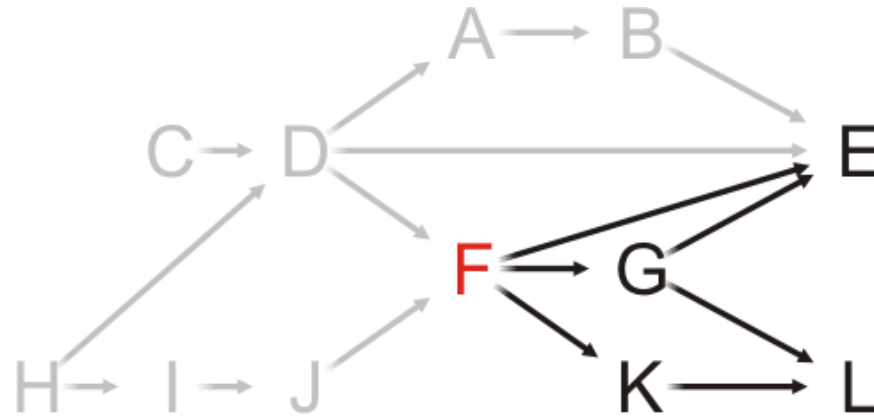
C, H, D, A, B, I

- Having completed Task J, what now?



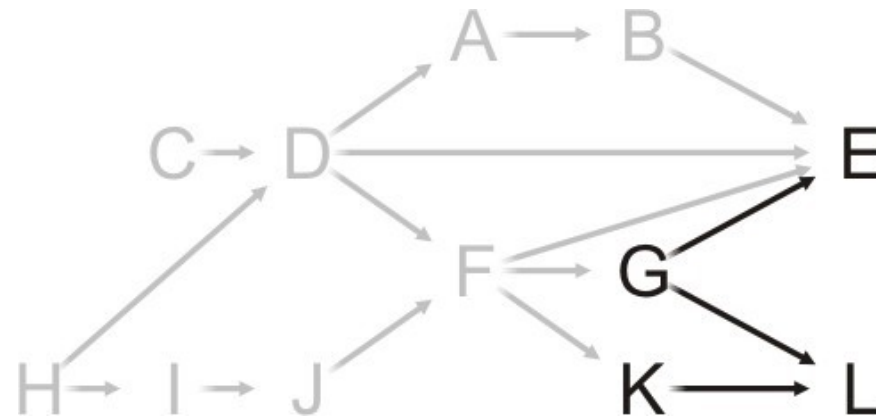
C, H, D, A, B, I, J

- Only Task F can be completed, so choose it



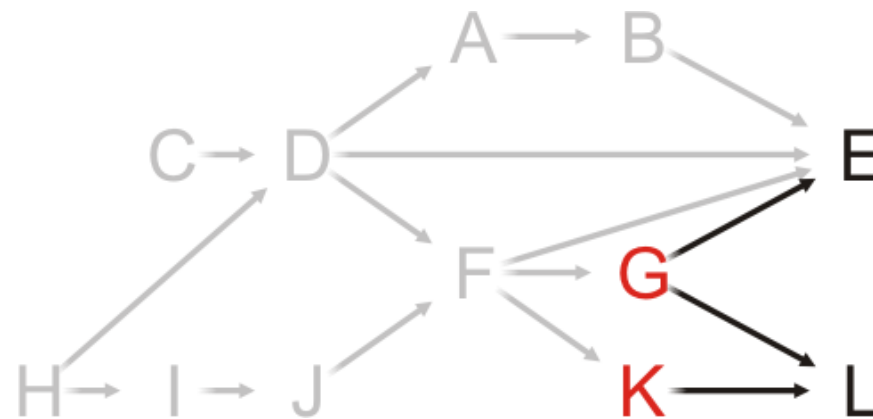
C, H, D, A, B, I, J

- What choices do we have now?



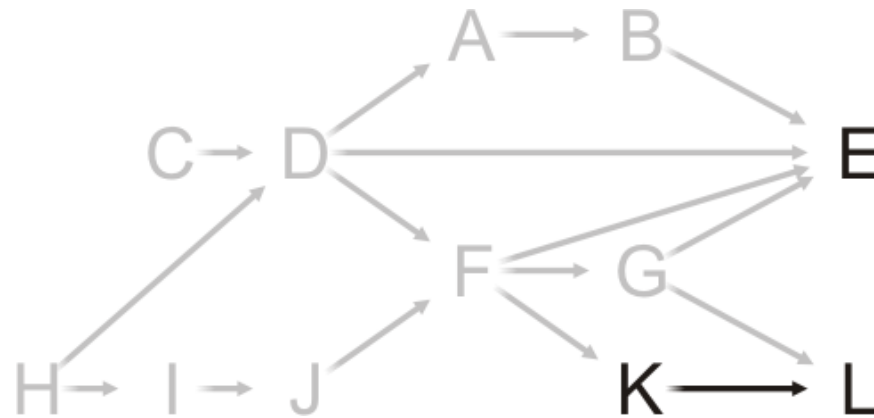
C, H, D, A, B, I, J, F

- We can perform Tasks G or K
- Choose Task G



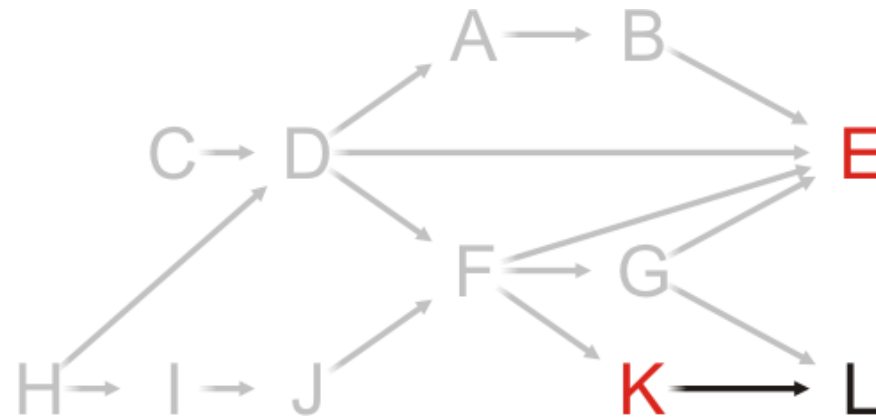
C, H, D, A, B, I, J, F

- Having removed Task G from the graph, what next?



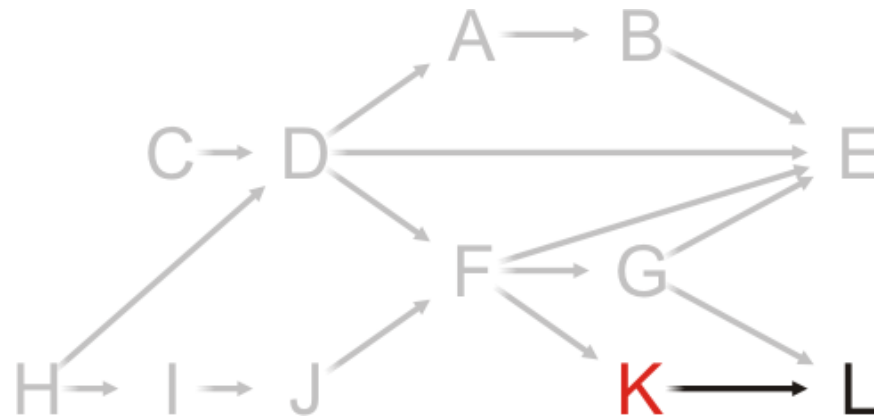
C, H, D, A, B, I, J, F, G

- Choosing between Tasks E and K, choose Task E



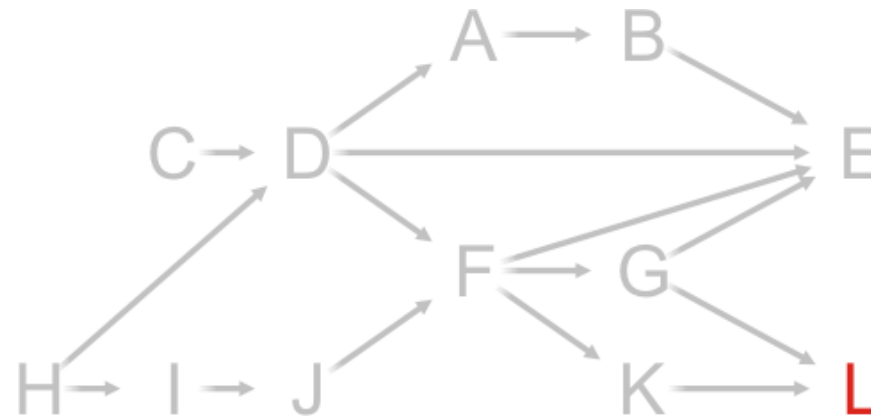
C, H, D, A, B, I, J, F, G

- At this point, Task K is the only one that can be run



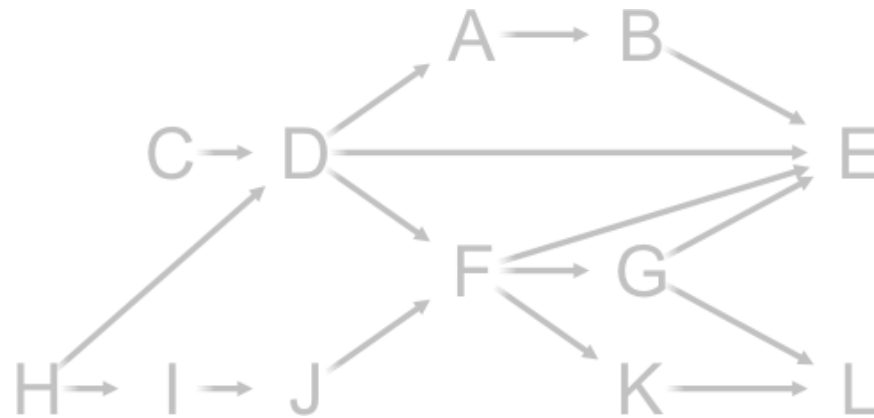
C, H, D, A, B, I, J, F, G, E

- And now that both Tasks G and K are complete, we can complete Task L



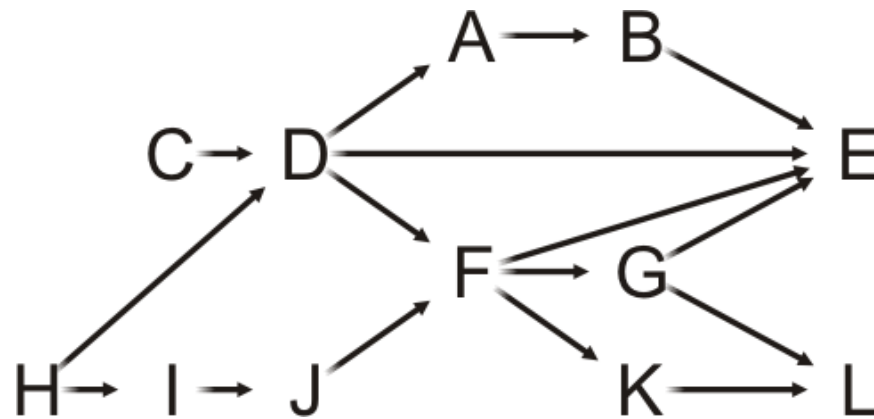
C, H, D, A, B, I, J, F, G, E, K

- There are no more vertices left



C, H, D, A, B, I, J, F, G, E, K, L

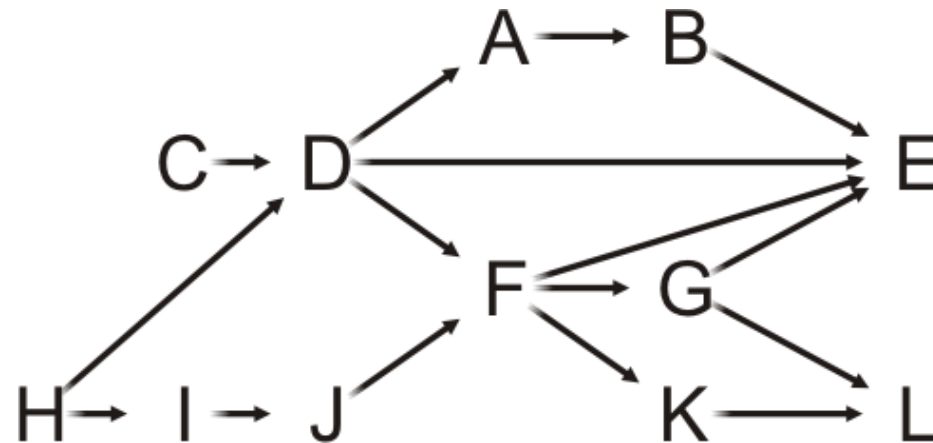
- Thus, one possible topological sort would be:
C, H, D, A, B, I, J, F, G, E, K, L



- Note that topological sorts need not be unique:

C, H, D, A, B, I, J, F, G, E, K, L

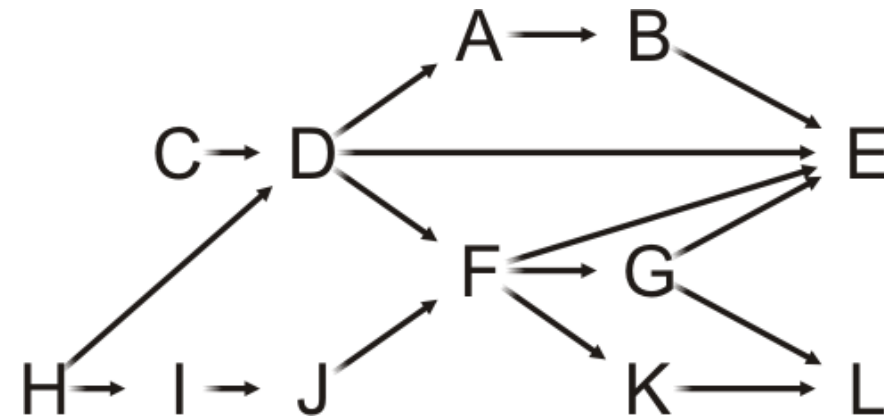
H, I, J, C, D, F, G, K, L, A, B, E



Analysis

- Firstly, find the **in-degrees** of each v .
 - Maintain a **table of the in-degrees** of each v .
 - Requires $O(?)$ memory.

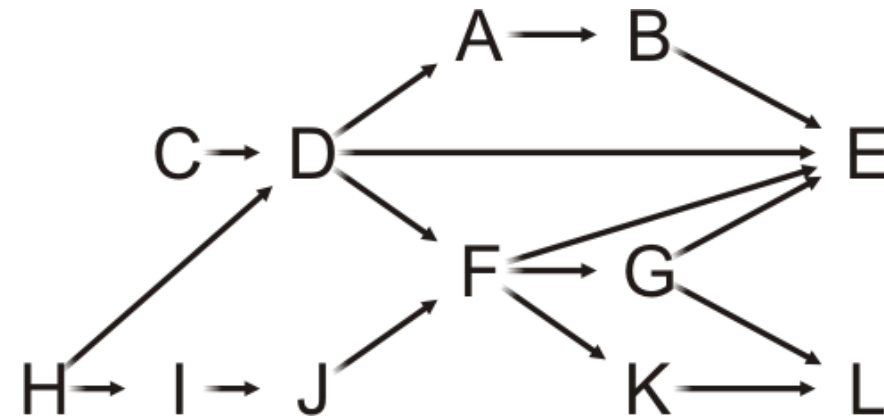
A	1
B	1
C	0
D	2
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2



Analysis

- Firstly, find the **in-degrees** of each v .
 - Maintain a **table of the in-degrees** of each v .
 - Requires $O(V)$ memory.

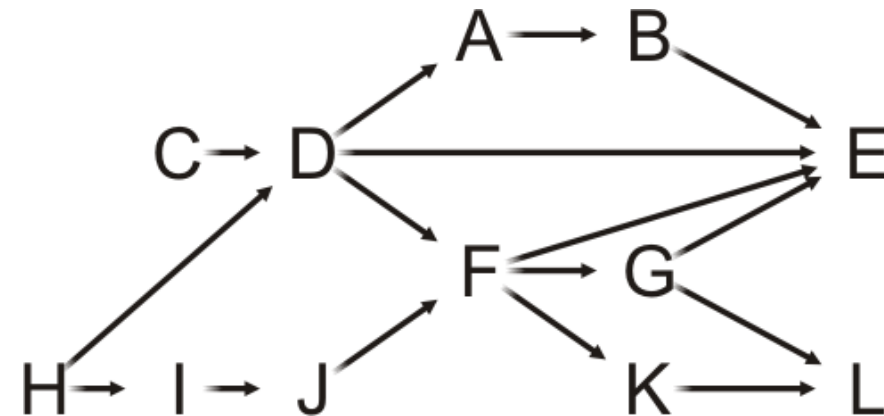
A	1
B	1
C	0
D	2
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2



Analysis

- Firstly, find the **in-degrees** of each v .
 - Maintain a **table of the in-degrees** of each v .
 - Requires $O(V)$ memory.
- **Idea:** after visiting a vertex, find next vertex with in-degree = 0:
 - Time complexity: $O(?)$

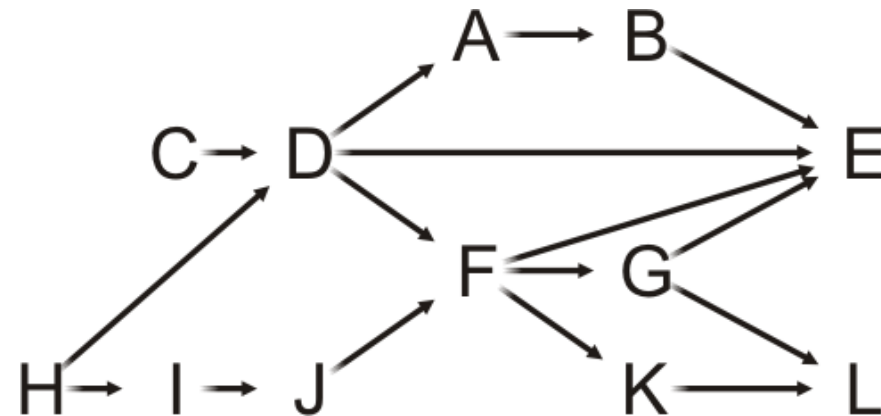
A	1
B	1
C	0
D	2
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2



Analysis

- Firstly, find the **in-degrees** of each v .
 - Maintain a **table of the in-degrees** of each v .
 - Requires $O(V)$ memory.
- **Idea:** after visiting a vertex, find next vertex with in-degree = 0:
 - Time complexity: $O(V^2)$

A	1
B	1
C	0
D	2
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2



- Better idea:

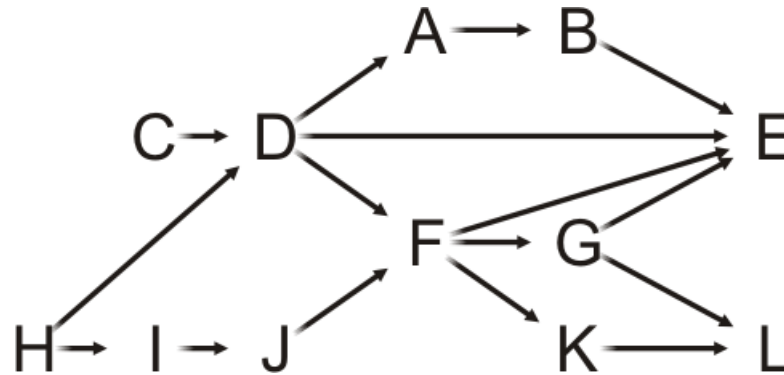
- Use **Queue** to temporarily store vertices with **in-degree zero**
- Each time in-degree of a vertex is **decremented to zero**, Enqueue it.

Topological Sort with Queue (Q)

- Initialization:
 - **Allocate memory** and **initialize** an array of in-degrees
 - Initialize Q with **all v having in-degree 0**
- **While Q is not empty:**
 - **Pop** a vertex from Q
 - **Decrement** the in-degree of each neighbor
 - Neighbors whose in-degree **becomes zero are pushed into Q**

Example

Stepping through the table, push all source vertices into the queue



A	1
B	1
C	0
D	2
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2

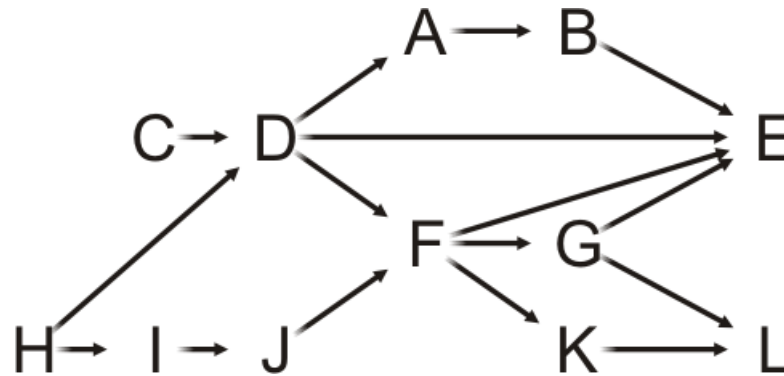
Queue:



The queue is empty

Example

Stepping through the table, push all source vertices into the queue

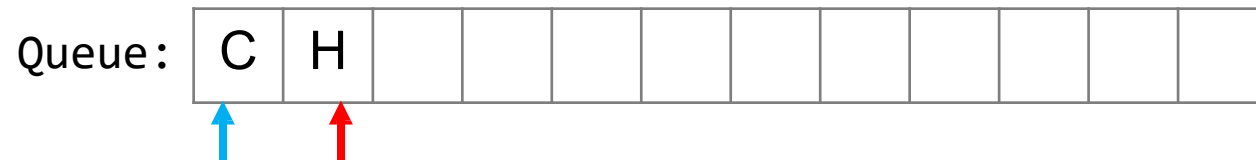
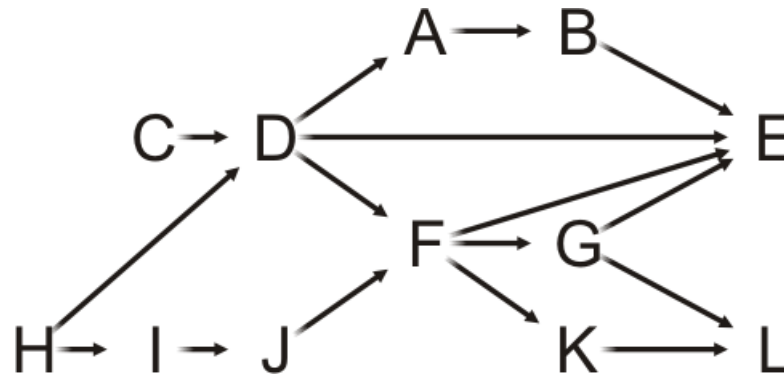


The queue is empty

A	1
B	1
C	0
D	2
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2

Example

Pop the front of the queue

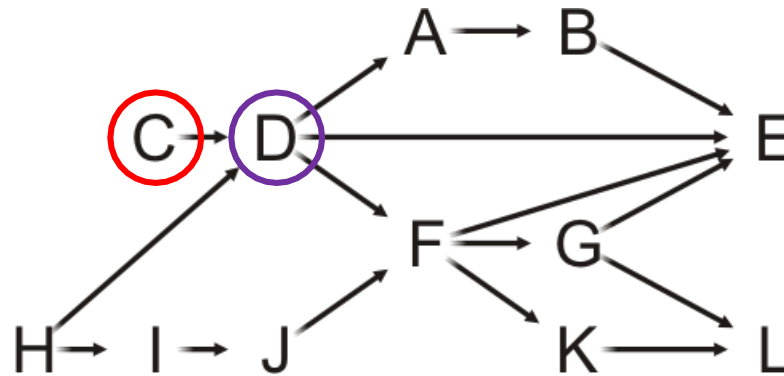


A	1
B	1
C	0
D	2
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2

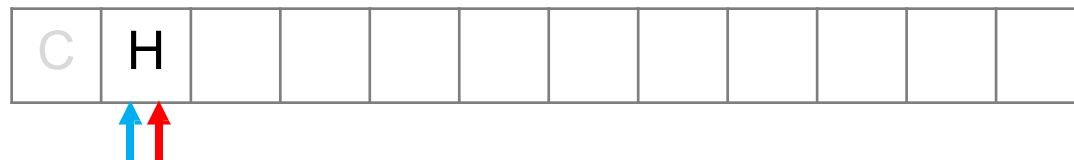
Example

Pop the front of the queue

- C has one neighbor: D



Queue:

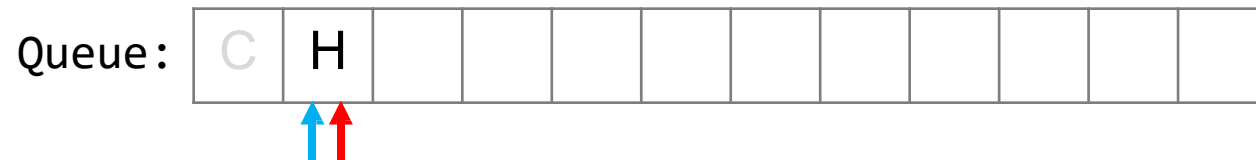
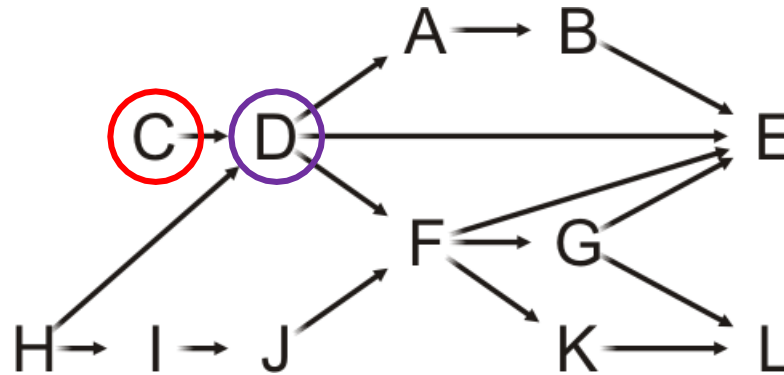


A	1
B	1
C	0
D	2
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2

Example

Pop the front of the queue

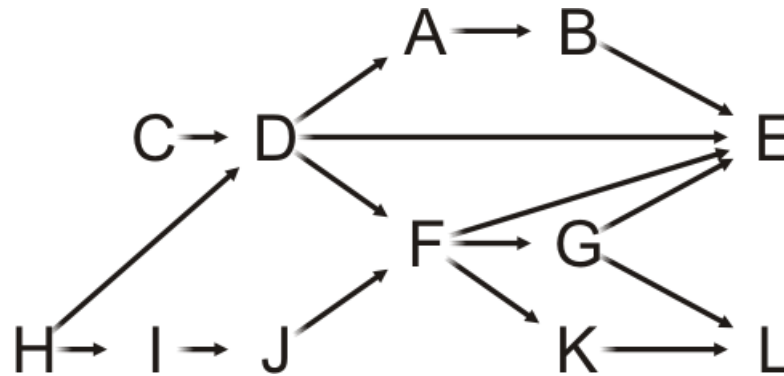
- C has one neighbor: D
- Decrement its in-degree



A	1
B	1
C	0
D	1
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2

Example

Pop the front of the queue



Queue:

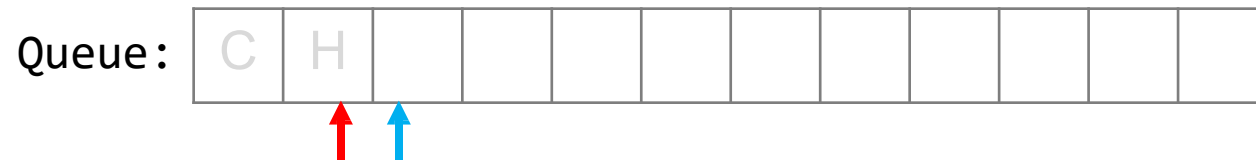
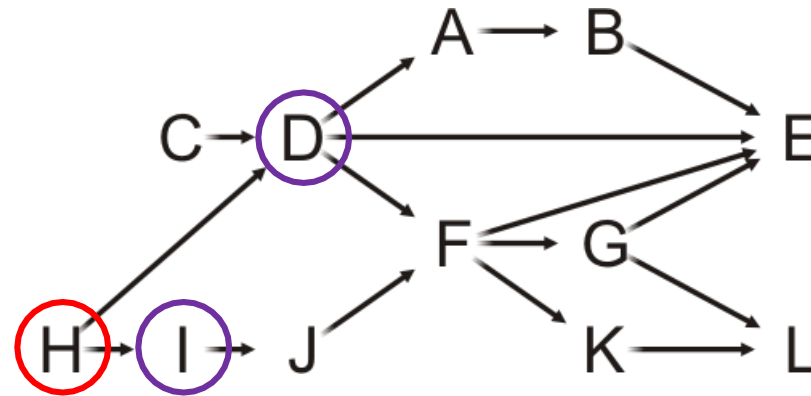


A	1
B	1
C	0
D	1
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2

Example

Pop the front of the queue

- H has two neighbors: D and I

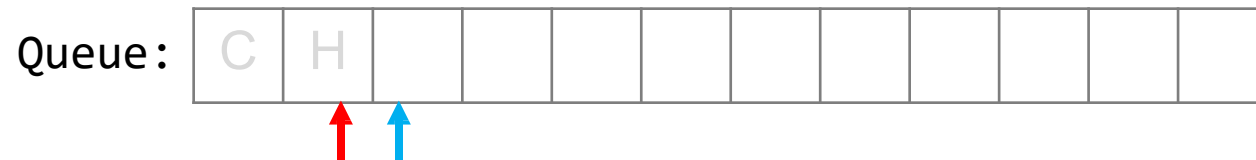
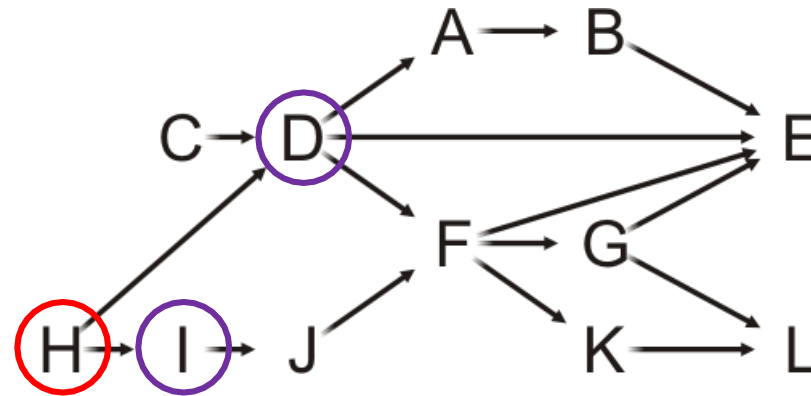


A	1
B	1
C	0
D	1
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2

Example

Pop the front of the queue

- H has two neighbors: D and I
- Decrement their in-degrees

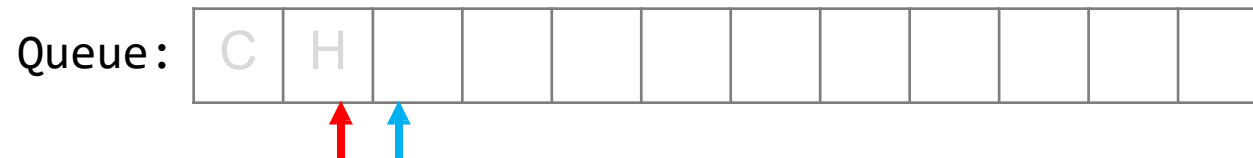
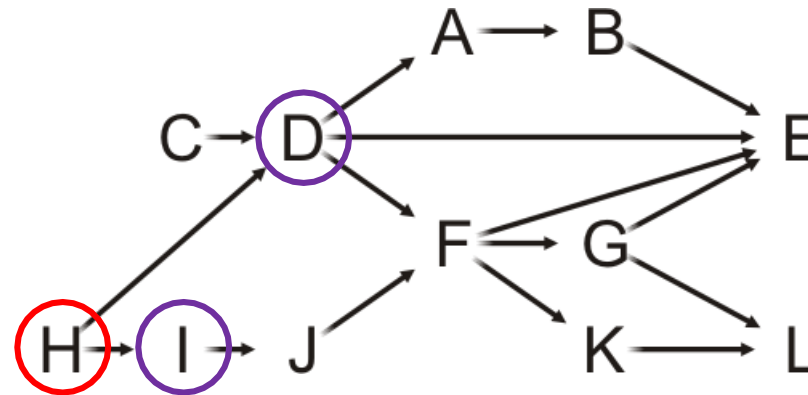


A	1
B	1
C	0
D	0
E	4
F	2
G	1
H	0
I	0
J	1
K	1
L	2

Example

Pop the front of the queue

- H has two neighbors: D and I
- Decrement their in-degrees
- Both are decremented to zero, so push them onto the queue

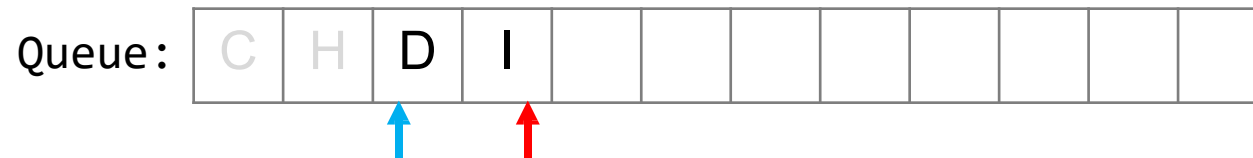
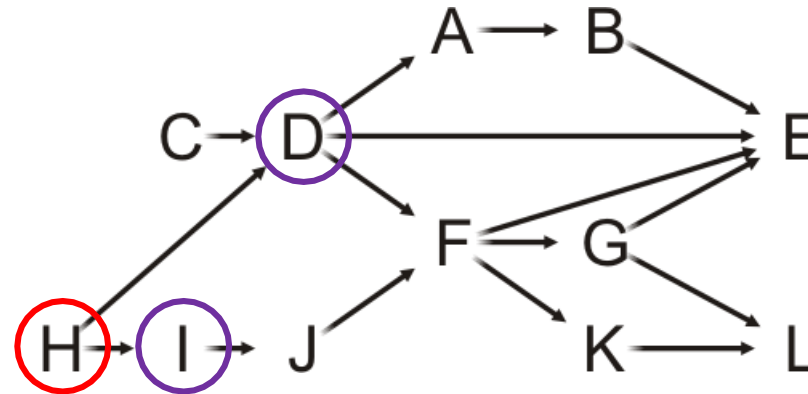


A	1
B	1
C	0
D	0
E	4
F	2
G	1
H	0
I	0
J	1
K	1
L	2

Example

Pop the front of the queue

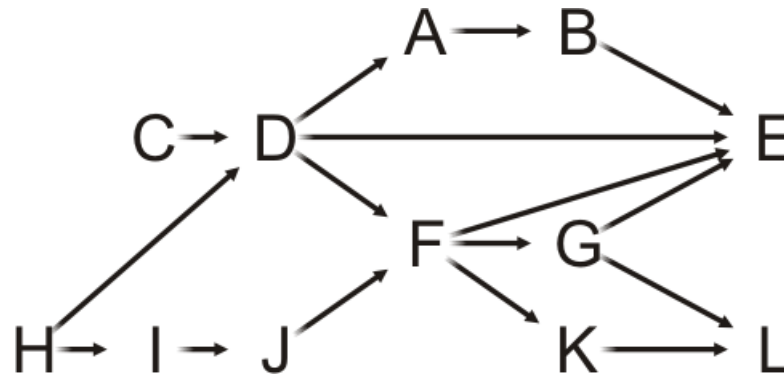
- H has two neighbors: D and I
- Decrement their in-degrees
- Both are decremented to zero, so push them onto the queue



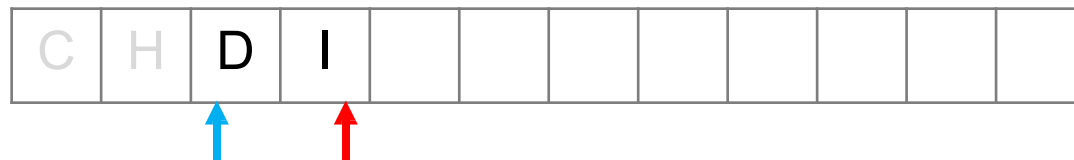
A	1
B	1
C	0
D	0
E	4
F	2
G	1
H	0
I	0
J	1
K	1
L	2

Example

Pop the front of the queue



Queue:

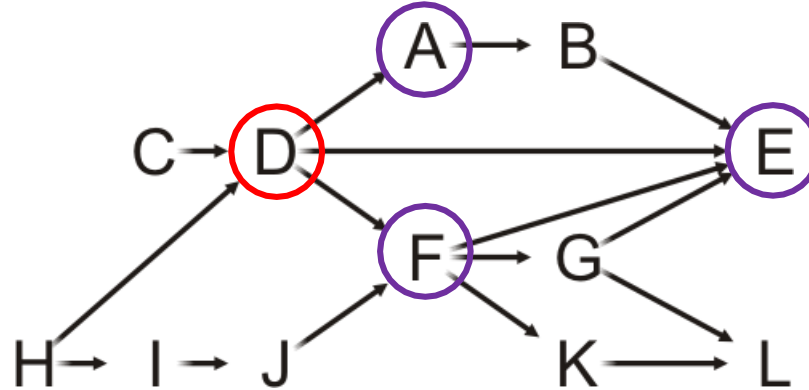


A	1
B	1
C	0
D	0
E	4
F	2
G	1
H	0
I	0
J	1
K	1
L	2

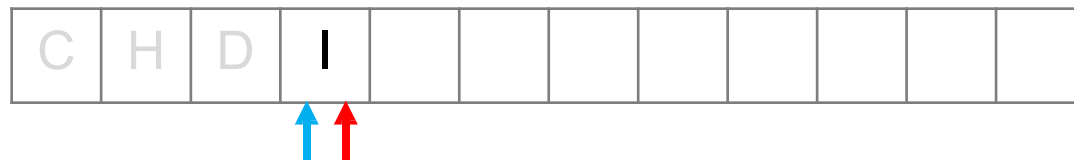
Example

Pop the front of the queue

- D has three neighbors: A, E and F



Queue:

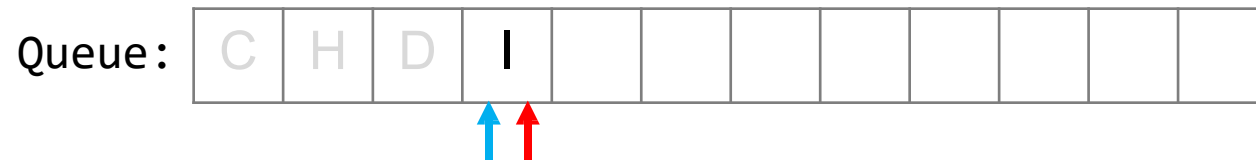
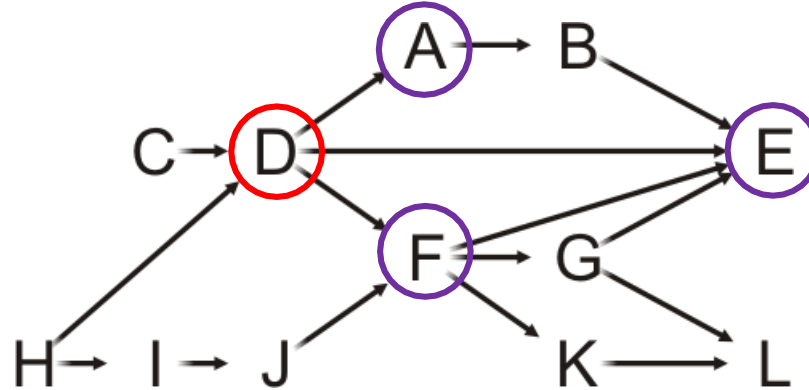


A	1
B	1
C	0
D	0
E	4
F	2
G	1
H	0
I	0
J	1
K	1
L	2

Example

Pop the front of the queue

- D has three neighbors: A, E and F
- Decrement their in-degrees

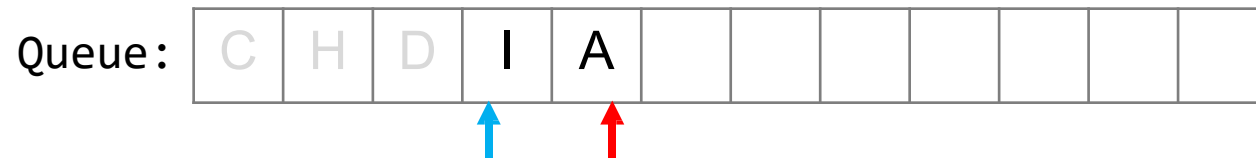
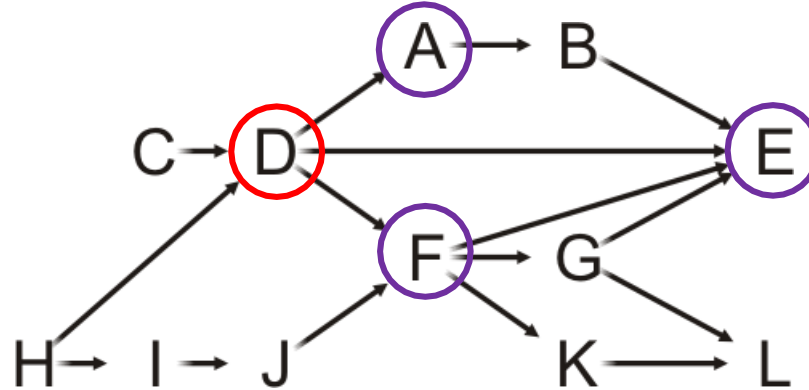


A	0
B	1
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	1
K	1
L	2

Example

Pop the front of the queue

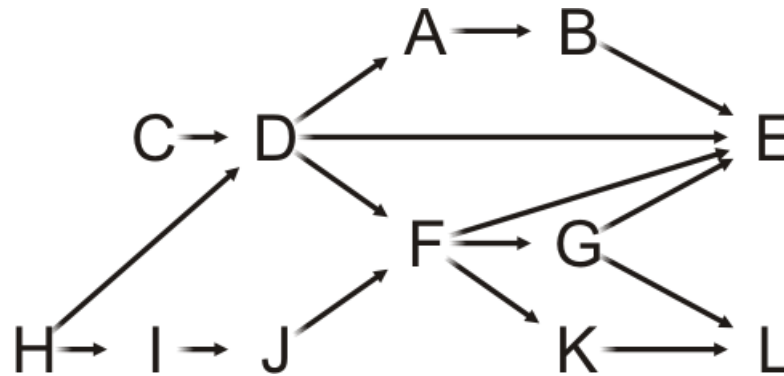
- D has three neighbors: A, E and F
- Decrement their in-degrees
 - A is decremented to zero, so push it onto the queue



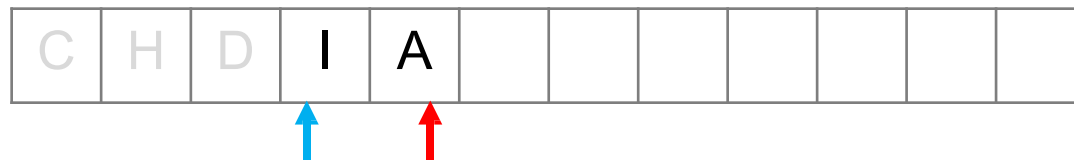
A	0
B	1
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	1
K	1
L	2

Example

Pop the front of the queue



Queue:

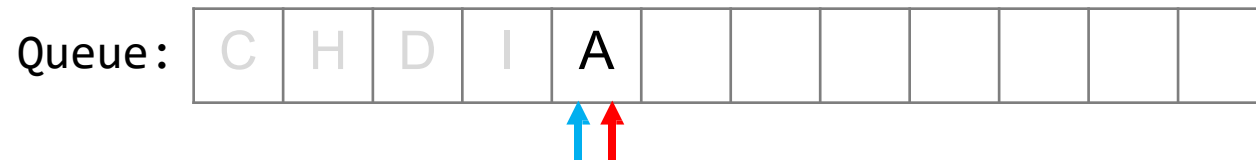
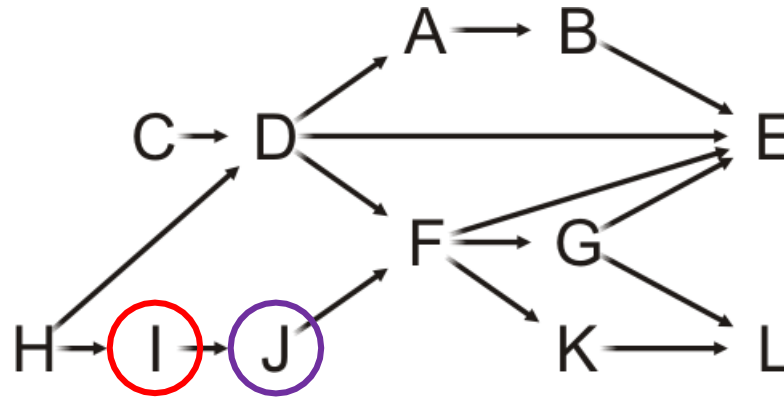


A	0
B	1
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	1
K	1
L	2

Example

Pop the front of the queue

- I has one neighbor: J

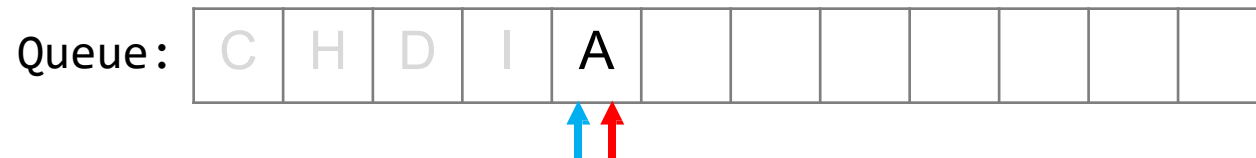
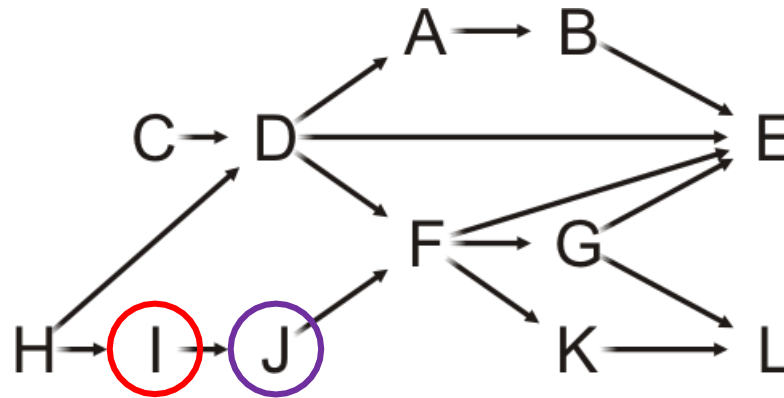


A	0
B	1
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	1
K	1
L	2

Example

Pop the front of the queue

- I has one neighbor: J
- Decrement its in-degree

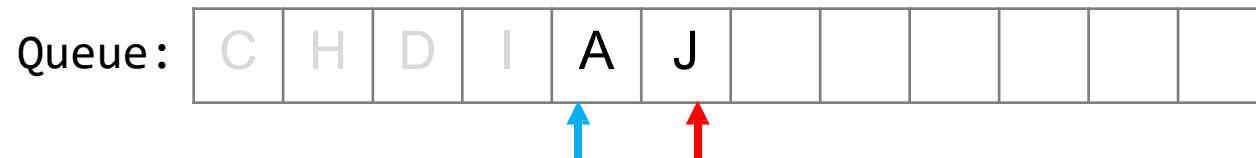
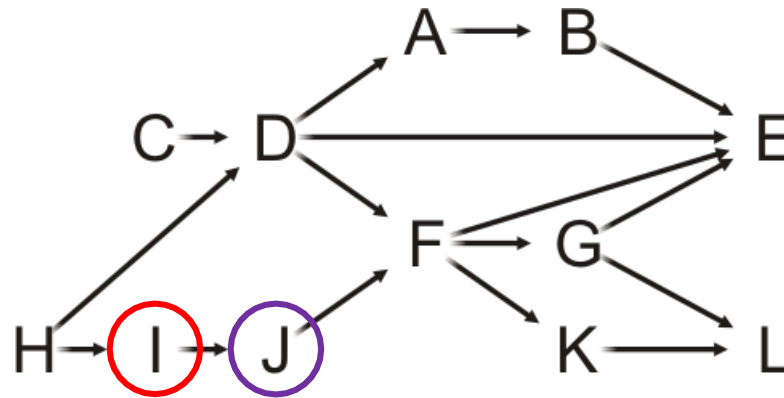


A	0
B	1
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue

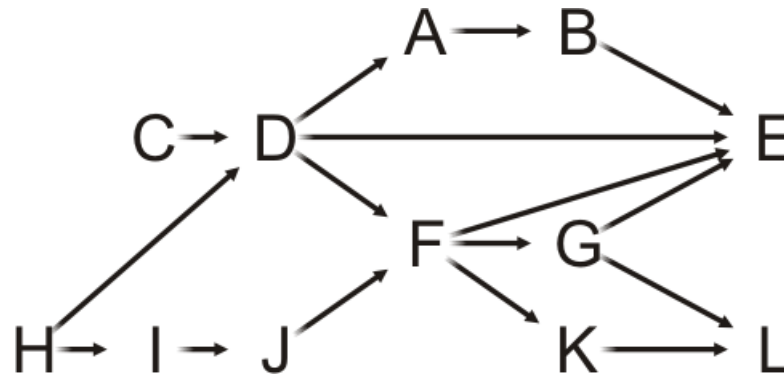
- I has one neighbor: J
- Decrement its in-degree
 - J is decremented to zero, so push it onto the queue



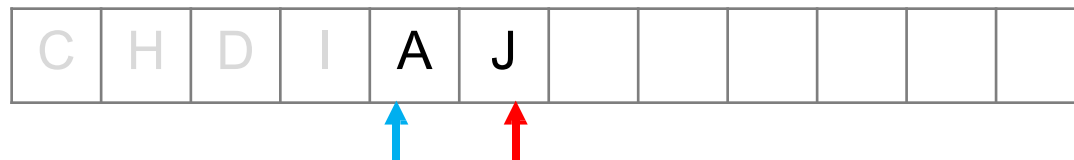
A	0
B	1
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue



Queue:

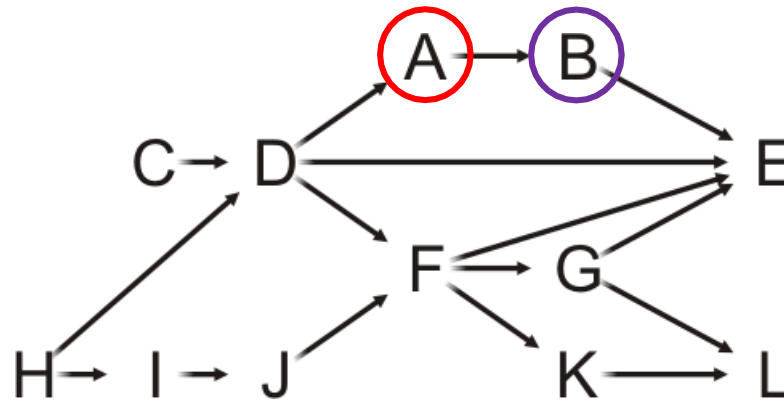


A	0
B	1
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	0
K	1
L	2

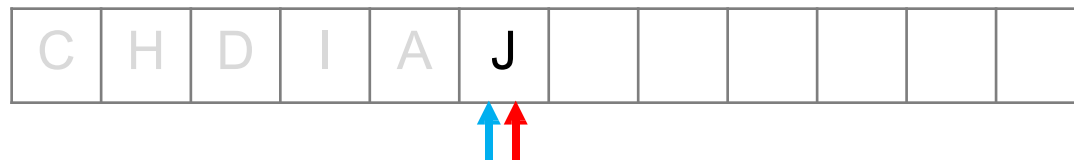
Example

Pop the front of the queue

- A has one neighbor: B



Queue:

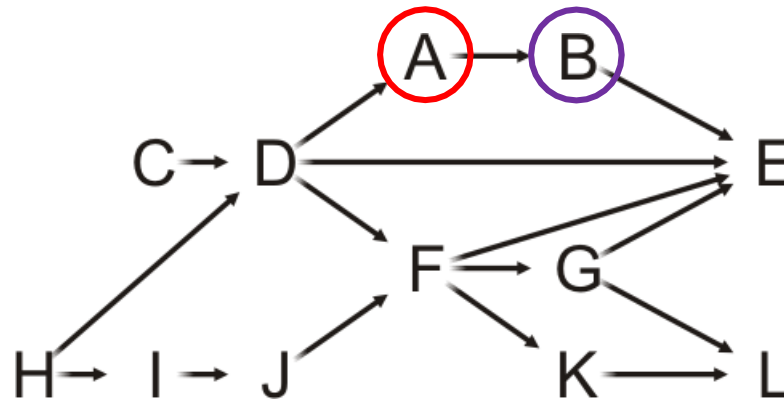


A	0
B	1
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue

- A has one neighbor: B
- Decrement its in-degree



Queue:

C	H	D	I	A	J						
---	---	---	---	---	---	--	--	--	--	--	--

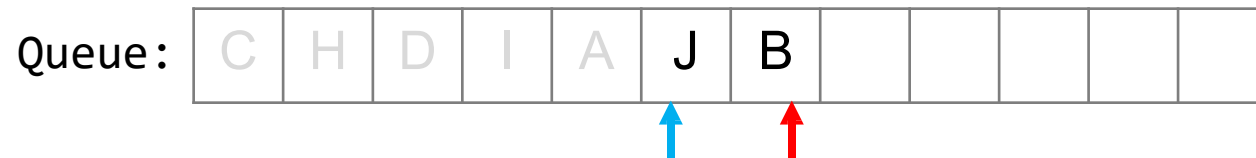
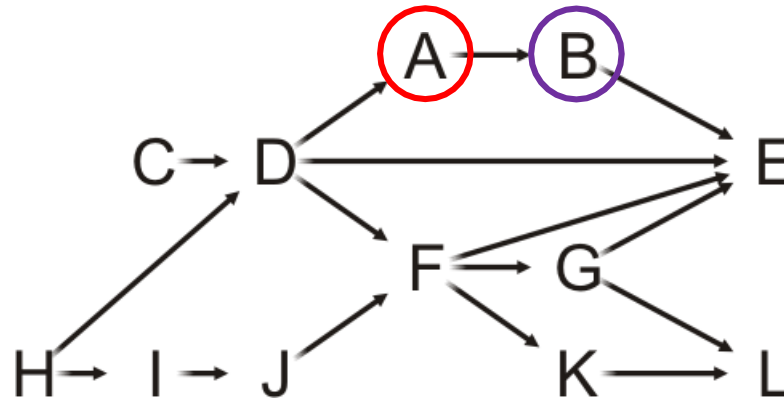


A	0
B	0
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue

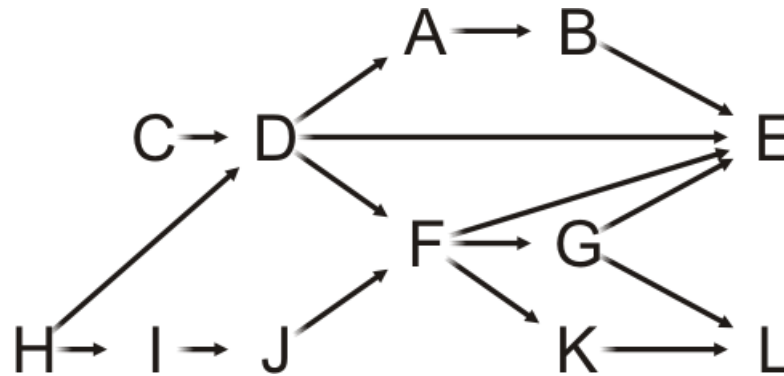
- A has one neighbor: B
- Decrement its in-degree
 - B is decremented to zero, so push it onto the queue



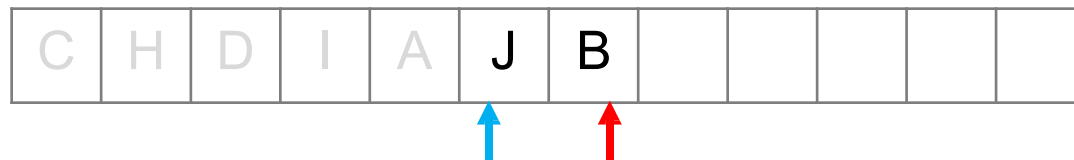
A	0
B	0
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue



Queue:

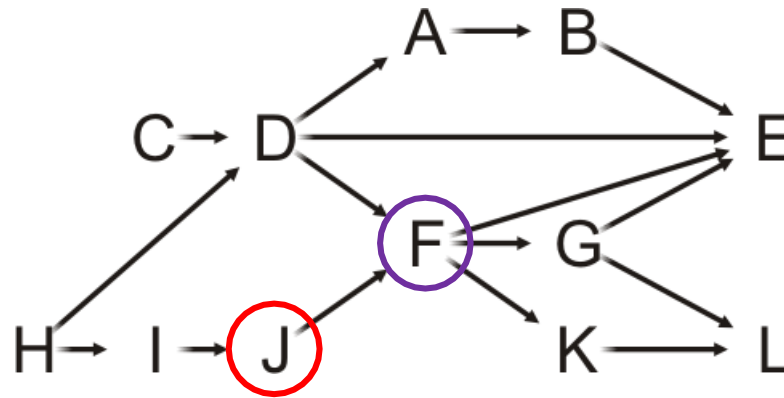


A	0
B	0
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue

- J has one neighbor: F

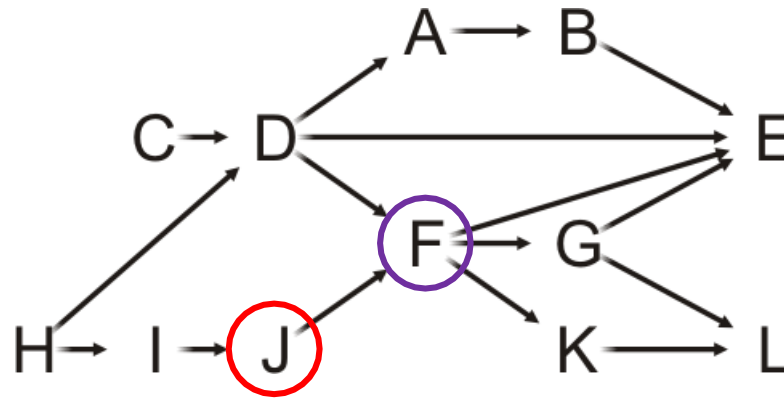


A	0
B	0
C	0
D	0
E	3
F	1
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue

- J has one neighbor: F
- Decrement its in-degree

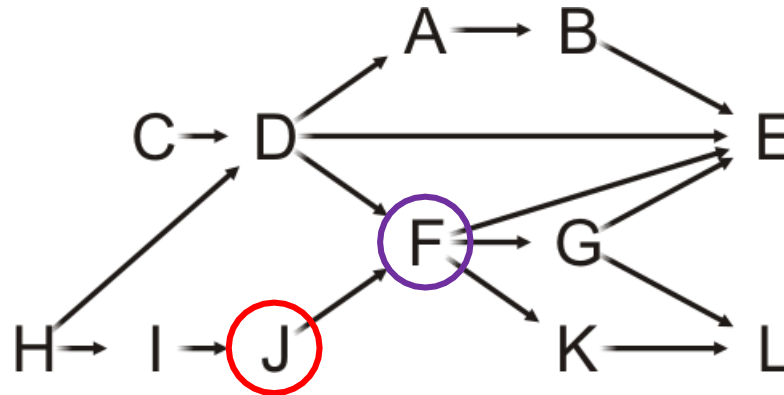


A	0
B	0
C	0
D	0
E	3
F	0
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue

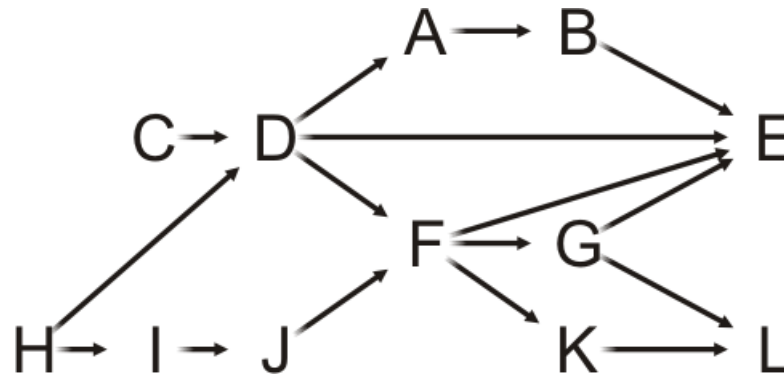
- J has one neighbor: F
- Decrement its in-degree
- F is decremented to zero, so push it onto the queue



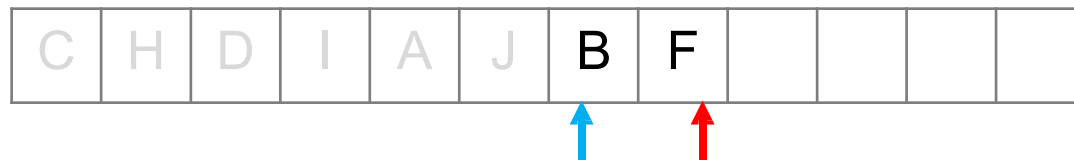
A	0
B	0
C	0
D	0
E	3
F	0
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue



Queue:

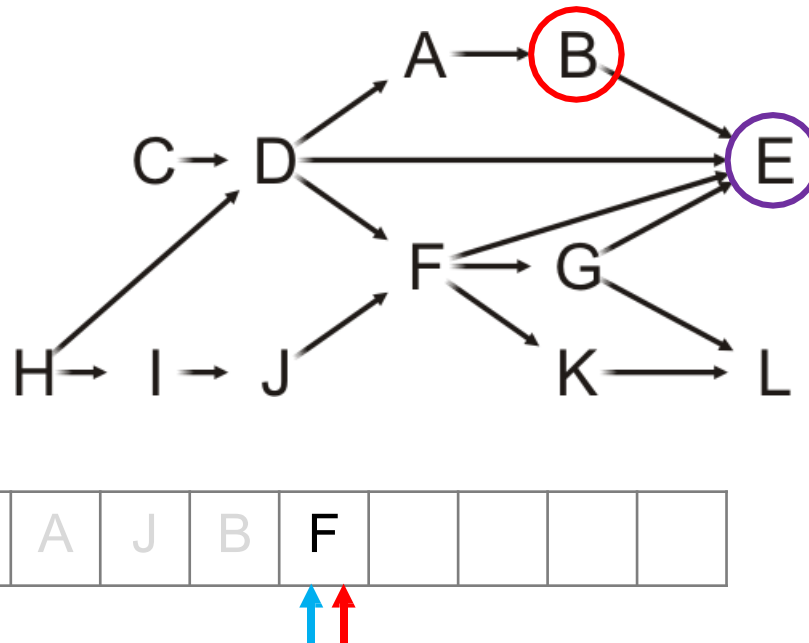


A	0
B	0
C	0
D	0
E	3
F	0
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue

- B has one neighbor: E



Queue:

C	H	D	I	A	J	B	F				
---	---	---	---	---	---	---	---	--	--	--	--

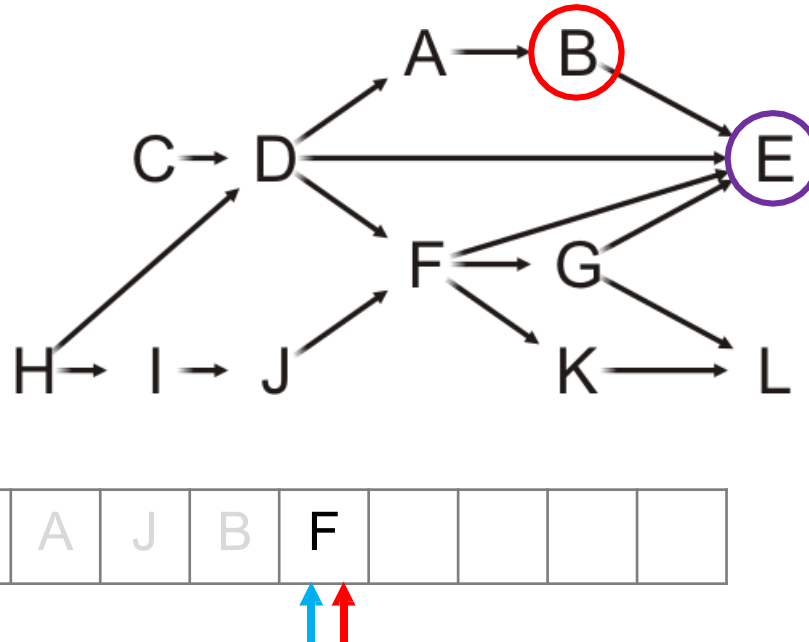


A	0
B	0
C	0
D	0
E	3
F	0
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue

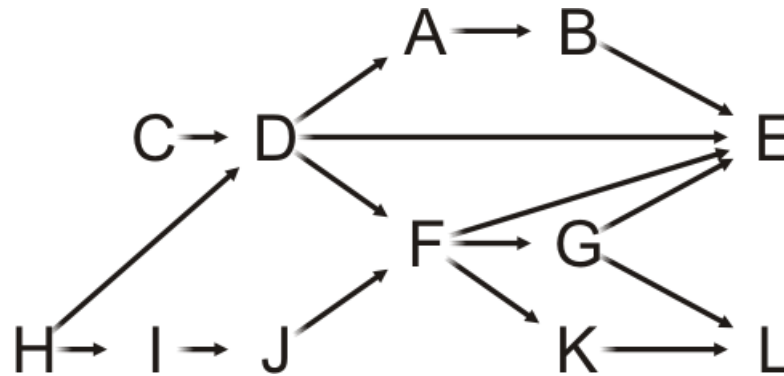
- B has one neighbor: E
- Decrement its in-degree



A	0
B	0
C	0
D	0
E	2
F	0
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue



Queue:

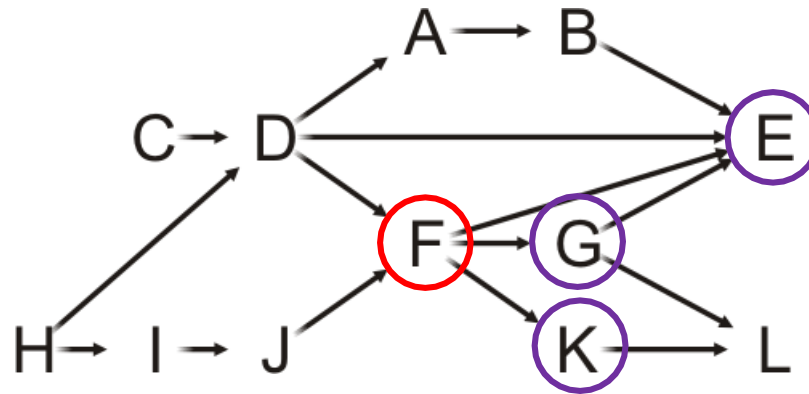


A	0
B	0
C	0
D	0
E	2
F	0
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue

- F has three neighbors: E, G and K



Queue:

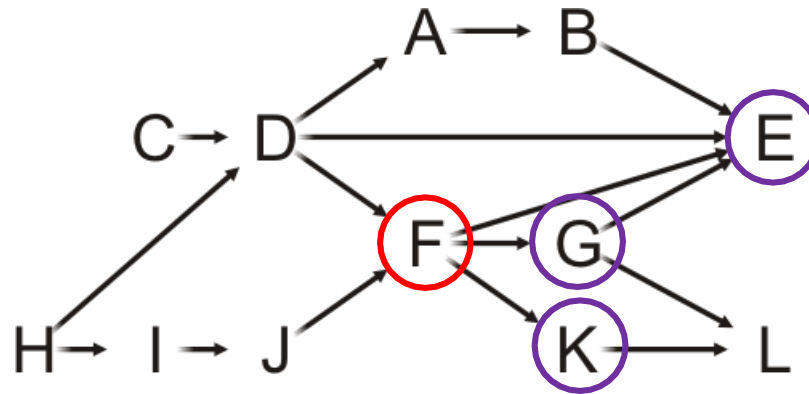


A	0
B	0
C	0
D	0
E	2
F	0
G	1
H	0
I	0
J	0
K	1
L	2

Example

Pop the front of the queue

- F has three neighbors: E, G and K
- Decrement their in-degrees



Queue:

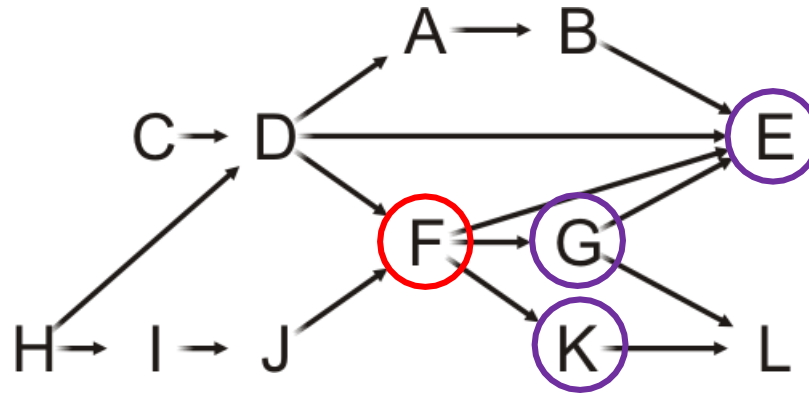


A	0
B	0
C	0
D	0
E	1
F	0
G	0
H	0
I	0
J	0
K	0
L	2

Example

Pop the front of the queue

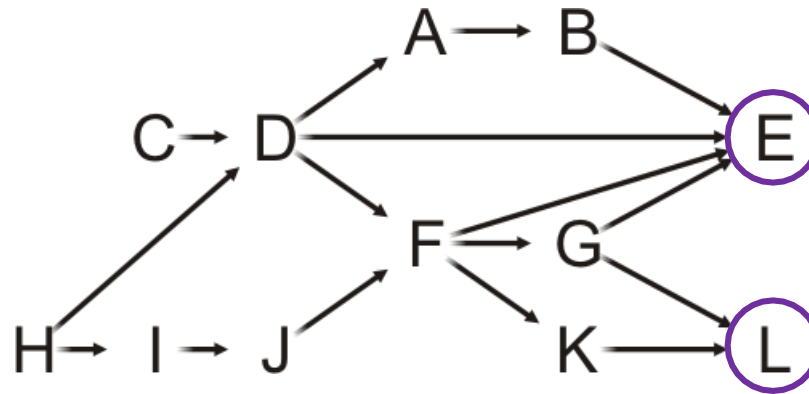
- F has three neighbors: E, G and K
- Decrement their in-degrees
 - G and K are decremented to zero, so push them onto the queue



A	0
B	0
C	0
D	0
E	1
F	0
G	0
H	0
I	0
J	0
K	0
L	2

Example

Pop the front of the queue



Queue:

C	H	D	I	A	J	B	F	G	K		
---	---	---	---	---	---	---	---	---	---	--	--

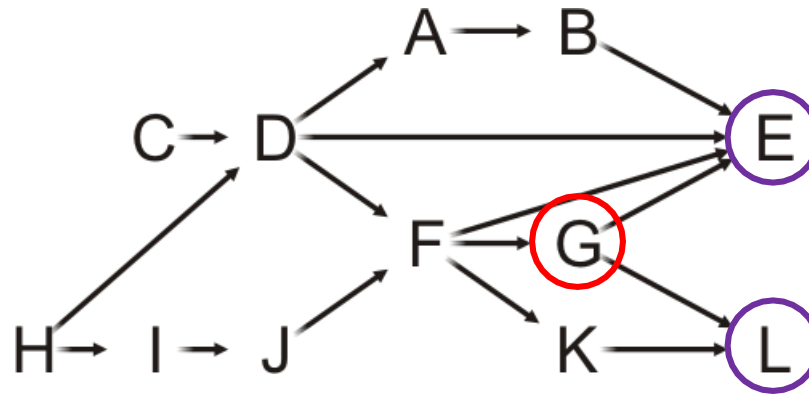


A	0
B	0
C	0
D	0
E	1
F	0
G	0
H	0
I	0
J	0
K	0
L	2

Example

Pop the front of the queue

- G has two neighbors: E and L



Queue:

C	H	D	I	A	J	B	F	G	K		
---	---	---	---	---	---	---	---	---	---	--	--

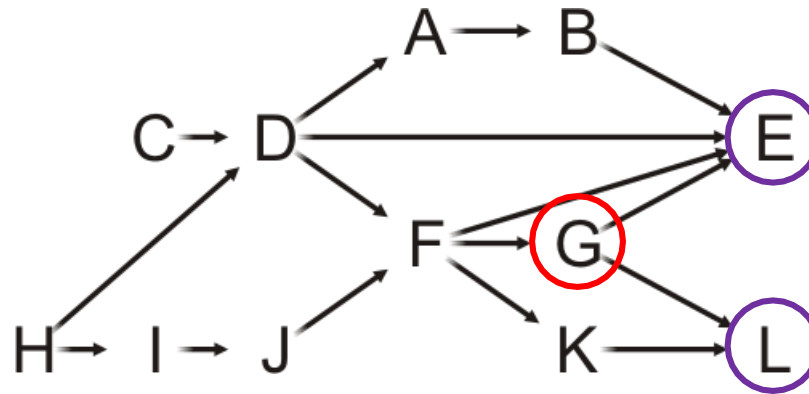


A	0
B	0
C	0
D	0
E	1
F	0
G	0
H	0
I	0
J	0
K	0
L	2

Example

Pop the front of the queue

- G has two neighbors: E and L
- Decrement their in-degrees



Queue:

C	H	D	I	A	J	B	F	G	K		
---	---	---	---	---	---	---	---	---	---	--	--

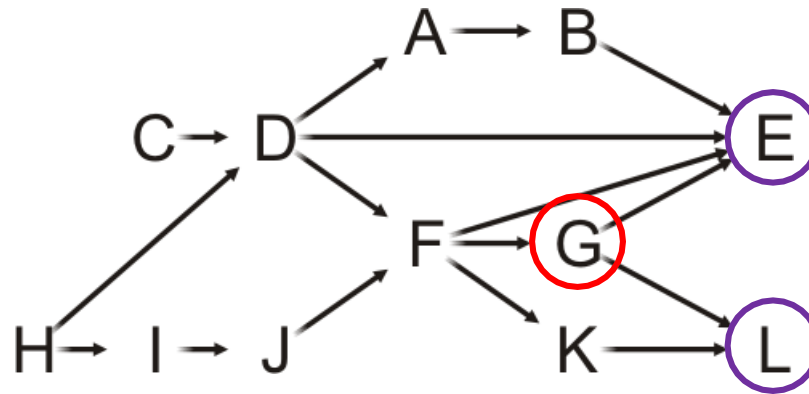


A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	1

Example

Pop the front of the queue

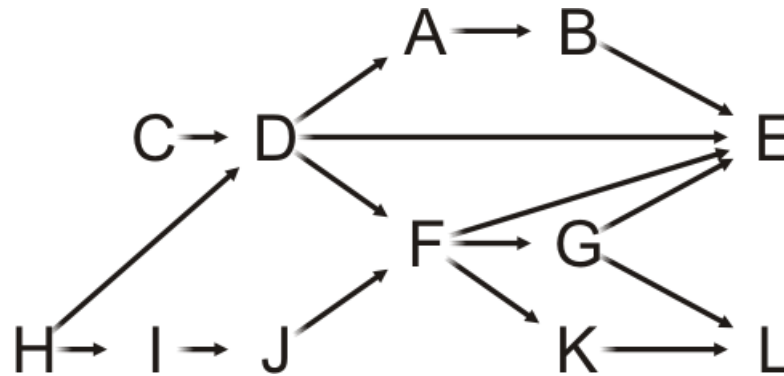
- G has two neighbors: E and L
- Decrement their in-degrees
 - E is decremented to zero, so push it onto the queue



A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	1

Example

Pop the front of the queue



Queue:

C	H	D	I	A	J	B	F	G	K	E	
---	---	---	---	---	---	---	---	---	---	---	--

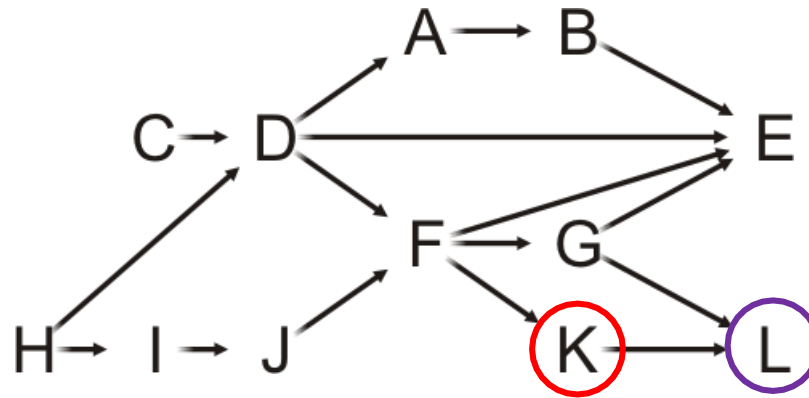


A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	1

Example

Pop the front of the queue

- K has one neighbors: L

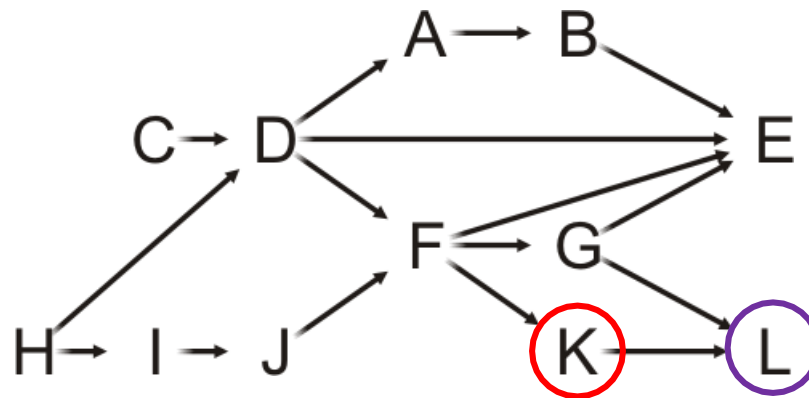


A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	1

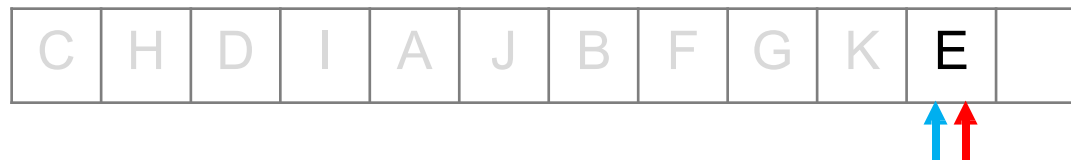
Example

Pop the front of the queue

- K has one neighbors: L
- Decrement its in-degree



Queue:

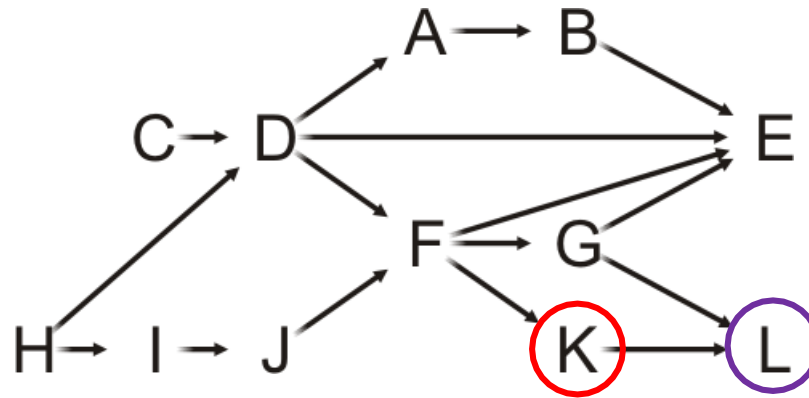


A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	0

Example

Pop the front of the queue

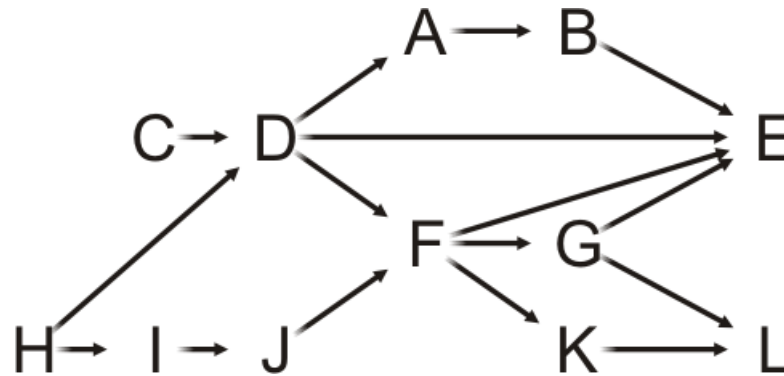
- K has one neighbors: L
- Decrement its in-degree
- L is decremented to zero, so push it onto the queue



A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	0

Example

Pop the front of the queue



Queue:

C	H	D	I	A	J	B	F	G	K	E	L
---	---	---	---	---	---	---	---	---	---	---	---

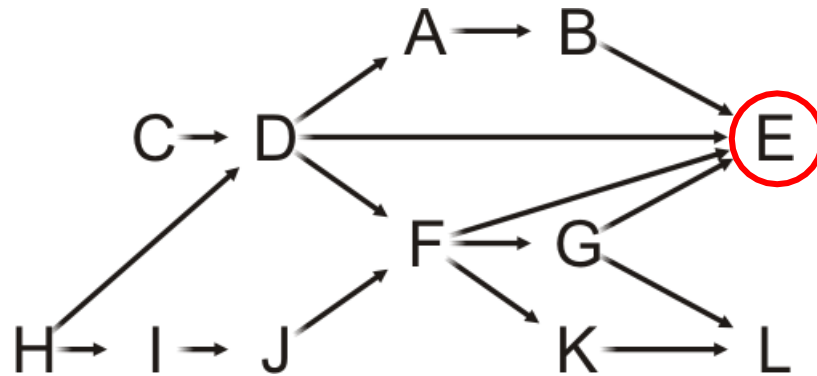


A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	0

Example

Pop the front of the queue

- E has no neighbors—it is a *sink*



Queue:

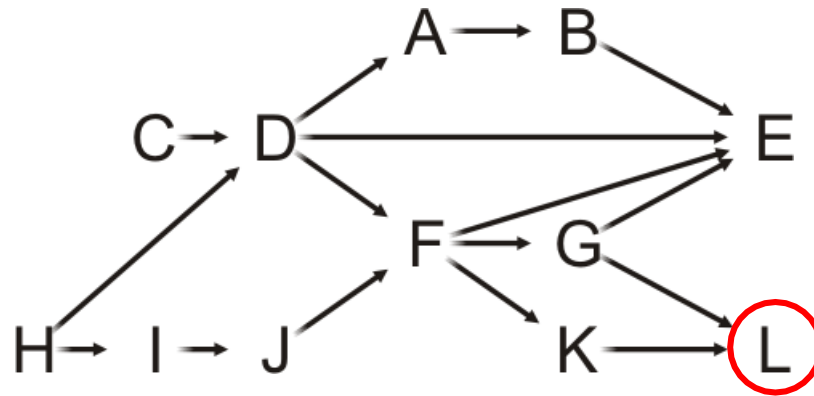
C	H	D	I	A	J	B	F	G	K	E	L
---	---	---	---	---	---	---	---	---	---	---	---



A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	0

Example

Pop the front of the queue



Queue:

C	H	D	I	A	J	B	F	G	K	E	L
---	---	---	---	---	---	---	---	---	---	---	---

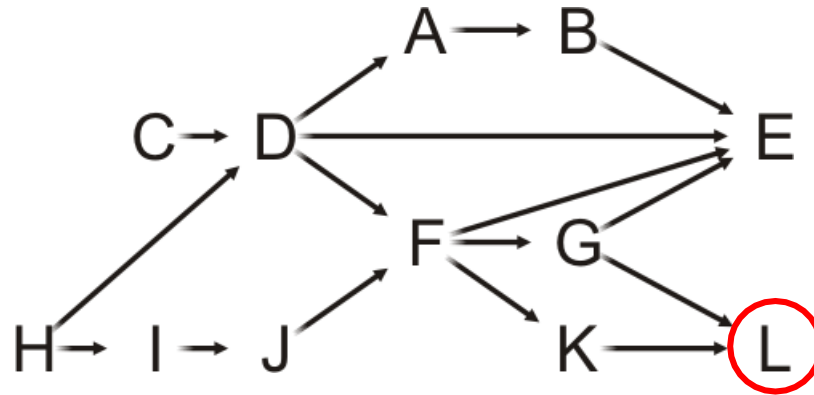


A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	0

Example

Pop the front of the queue

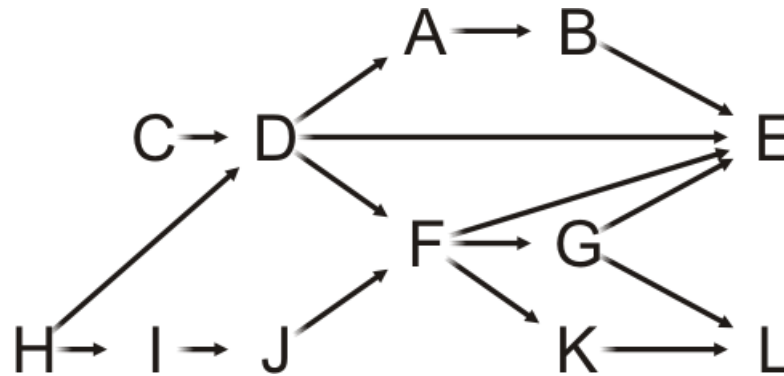
- L has no neighbors—it is also a *sink*



A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	0

Example

The queue is empty, so we are done



Queue:

C	H	D	I	A	J	B	F	G	K	E	L
---	---	---	---	---	---	---	---	---	---	---	---

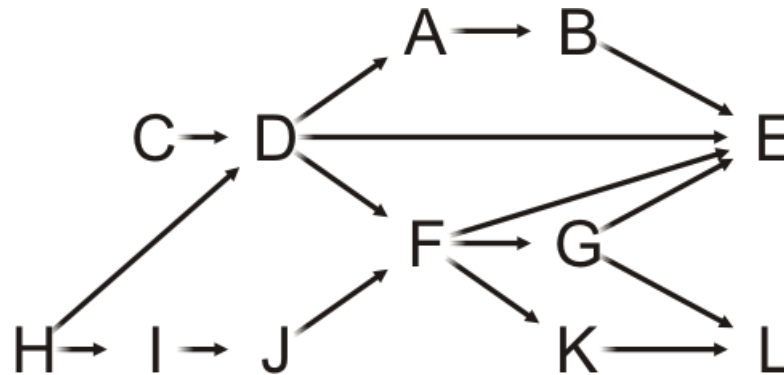


A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	0

Example

We deallocate the memory for the temporary in-degree array

The array stores the topological sorting



C	H	D	I	A	J	B	F	G	K	E	L
---	---	---	---	---	---	---	---	---	---	---	---

A	0
B	0
C	0
D	0
E	0
F	0
G	0
H	0
I	0
J	0
K	0
L	0

- Time complexity:

- Calculate the in-degree of each v once: $O(?)$
- For each vertex, need to consider the associated edges once.
 - $O(?)$: adjacency list
 - $O(?)$: adjacency matrix

Task	If stored using Adj. Matrix	If stored using Adj. List
Initialization: - Allocate memory and initialize an array of in-degrees - Initialize Q with all v having in-degree 0	Cost to initialize in-degrees = $O(V^2)$	Cost to initialize = $O(V + E)$
While Q is not empty : - Pop a vertex from Q - Decrement the in-degree of each neighbor - Neighbors whose in-degree becomes 0 are pushed into Q	- For each v, visit the neighbors. - No easy way ! - Check all entries of that row. - Cost of while loop = $O(V \times V) = O(V^2)$	- Just check dedicated list for that v ! - Each vertex will come in the Queue $O(V)$ - For each vertex, visit the neighbours resulting in $O(E)$ operations in total - So total loop costs $O(V + E)$
	Total cost = $O(V^2)$	Total Cost = $O(V + E)$

- Time complexity:
 - Calculate the in-degree of each v once: $O(V)$
 - For each vertex, need to consider the associated edges once.
 - $O(V + E)$: adjacency list
 - $O(V^2)$: adjacency matrix

- What happens if at some step, all remaining vertices have an in-degree > 0 ?
- Consequence: ??

- What happens if at some step, all remaining vertices have an in-degree > 0 ?
 - Consequence: we get $O(V + E)$ algorithm for **cycle detection**.

Thank you